DS3_to_A2

July 2, 2026

0.1 ## Adaptación del Dataset - DS3 a las caracteristicas del Articulo 2

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import ipywidgets as widgets
import tempfile
from IPython.display import display, clear_output
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, confusion_matrix, classification_report
from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler
import warnings
warnings.filterwarnings('ignore')
```

[2]:
```
# Descarga el dataset desde Kaggle:
# https://www.kaggle.com/datasets/jaison14/
 ↪electrical-fault-detection-and-classification
# Usar el directorio content de Colab
BASE_DIR = "/content"
UPLOAD_DIR = os.path.join(BASE_DIR, "archivos_subidos")


# Widget para subir archivos
uploader = widgets.FileUpload(
    accept='*',
    multiple=True,
    description='Upload Files'
)
output = widgets.Output()

def on_upload_change(change):
    with output:
        clear_output(wait=True)
```

```
        if not uploader.value:
            print("No hay archivos seleccionados.")
            return

        # Crear la carpeta (y todos los padres) sin errores si ya existe
        os.makedirs(UPLOAD_DIR, exist_ok=True)
        print(f"Carpeta lista: {UPLOAD_DIR}")

        # Guardar (o sobrescribir) los archivos
        for filename, file_info in uploader.value.items():
            content = file_info['content']
            file_path = os.path.join(UPLOAD_DIR, filename)
            with open(file_path, 'wb') as f:
                f.write(content)
            print(f" {filename} guardado en {file_path}")


        os.chdir(UPLOAD_DIR)
        print(f"Directorio actual cambiado a: {os.getcwd()}")
        # Mostrar solo los archivos cargados
        print("\nArchivos subidos:")
        for filename in uploader.value.keys():
            ruta_completa = os.path.join(UPLOAD_DIR, filename)
            print(f"- El archivo: {filename} \n  Se encuentra en la ruta: ->␣
 ↪{ruta_completa}")


uploader.observe(on_upload_change, names='value')

display(uploader)
display(output)
```

```
FileUpload(value={}, accept='*', description='Upload Files', multiple=True)
```

```
Output()
```

[3]:
```
# Cargar datos (ajusta la ruta si es necesario)
df = pd.read_csv('detect_dataset.csv')

# Ver primeras filas
print("Primeras 5 filas:")
print(df.head())

# Información general
print("\nInformación del dataset:")
print(df.info())
```

```python
# Estadísticas básicas
print("\nEstadísticas descriptivas:")
print(df.describe())

# Verificar valores nulos
print("\nValores nulos por columna:")
print(df.isnull().sum())

# Distribución de la variable objetivo
print("\nDistribución de 'Output (S)':")
print(df['Output (S)'].value_counts(normalize=True))
```

```
Primeras 5 filas:
   Output (S)          Ia        Ib          Ic        Va        Vb        Vc  \
0           0 -170.472196  9.219613  161.252583  0.054490 -0.659921  0.605431
1           0 -122.235754  6.168667  116.067087  0.102000 -0.628612  0.526202
2           0  -90.161474  3.813632   86.347841  0.141026 -0.605277  0.464251
3           0  -79.904916  2.398803   77.506112  0.156272 -0.602235  0.445963
4           0  -63.885255  0.590667   63.294587  0.180451 -0.591501  0.411050

   Unnamed: 7  Unnamed: 8
0         NaN         NaN
1         NaN         NaN
2         NaN         NaN
3         NaN         NaN
4         NaN         NaN

Información del dataset:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 12001 entries, 0 to 12000
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Output (S)  12001 non-null  int64
 1   Ia          12001 non-null  float64
 2   Ib          12001 non-null  float64
 3   Ic          12001 non-null  float64
 4   Va          12001 non-null  float64
 5   Vb          12001 non-null  float64
 6   Vc          12001 non-null  float64
 7   Unnamed: 7  0 non-null      float64
 8   Unnamed: 8  0 non-null      float64
dtypes: float64(8), int64(1)
memory usage: 843.9 KB
None

Estadísticas descriptivas:
```

```
         Output (S)            Ia            Ib            Ic            Va  \
count  12001.000000  12001.000000  12001.000000  12001.000000  12001.000000
mean       0.457962      6.709369    -26.557793     22.353043      0.010517
std        0.498250    377.158470    357.458613    302.052809      0.346221
min        0.000000   -883.542316   -900.526951   -883.357762     -0.620748
25%        0.000000    -64.348986    -51.421937    -54.562257     -0.237610
50%        0.000000     -3.239788      4.711283     -0.399419      0.002465
75%        1.000000     53.823453     69.637787     45.274542      0.285078
max        1.000000    885.738571    889.868884    901.274261      0.609864

                Vb            Vc  Unnamed: 7  Unnamed: 8
count  12001.000000  12001.000000         0.0         0.0
mean      -0.015498      0.004980         NaN         NaN
std        0.357644      0.349272         NaN         NaN
min       -0.659921     -0.612709         NaN         NaN
25%       -0.313721     -0.278951         NaN         NaN
50%       -0.007192      0.008381         NaN         NaN
75%        0.248681      0.289681         NaN         NaN
max        0.627875      0.608243         NaN         NaN

Valores nulos por columna:
Output (S)        0
Ia                0
Ib                0
Ic                0
Va                0
Vb                0
Vc                0
Unnamed: 7    12001
Unnamed: 8    12001
dtype: int64

Distribución de 'Output (S)':
Output (S)
0    0.542038
1    0.457962
Name: proportion, dtype: float64
```

0.2 Adaptación del Dataset - DS3 a las caracteristicas del Articulo 2

Para replicar fielmente la metodología del artículo 2, **no podemos usar una corriente de fase aislada (`Ia`) como variable objetivo**, porque el artículo predice la **potencia activa global** (`Global_active_power`) y utiliza la **intensidad global** (`Global_intensity`) como una característica más.

Los datos que tenemos (`Va, Vb, Vc, Ia, Ib, Ic`) nos permiten **calcular** exactamente esas magnitudes globales, emulando el medidor principal de la vivienda francesa.

0.2.1 Sobre la variable `Output (S)` del dataset original

La columna `Output (S)` es la **variable objetivo para clasificación de fallas** del dataset DS3[electrical-fault-detection-and-classification].

Indica si los datos corresponden a un estado **normal** (`0`) o a una **falla** (`1`). El dataset fue diseñado para **clasificación**, no para predicción de series temporales.

Esta variable NO se usa en nuestro análisis porque:

1. **Es una etiqueta de clasificación**, no una variable de entrada para predicción de consumo.
2. **Causaría data leakage**: el modelo aprendería la relación directa entre `Output (S)` y las variables eléctricas.
3. **No tiene sentido físico** para predecir consumo energético.
4. El dataset contiene **mediciones instantáneas**, no una serie temporal continua de consumo horario.

Por lo tanto, `Output (S)` se excluye completamente del pipeline de predicción y detección de anomalías.

0.2.2 Variables seleccionadas para el modelo

Vamos a redefinir las variables de esta manera (obteniendo exactamente **8 features**):

1. **Target (lo que vamos a predecir):**
   `Global_active_power` =

   $$|V_a \cdot I_a| + |V_b \cdot I_b| + |V_c \cdot I_c|$$

   (Potencia total absoluta, para evitar que corrientes negativas anulen el consumo).

2. **Feature 1: Voltaje RMS**
   En lugar de promediar los valores instantáneos (que pueden ser negativos y se cancelan en sistemas balanceados), calculamos el **voltaje RMS** de las tres fases:

   $$Voltage = \sqrt{\frac{V_a^2 + V_b^2 + V_c^2}{3}}$$

   Este valor **siempre es positivo** y es la magnitud físicamente correcta para representar la tensión del sistema (equivalente al valor que mediría un voltímetro).
   Nota: El promedio simple de las tres fases da valores cercanos a cero, lo que no aporta información útil al modelo.

3. **Feature 2: Intensidad Global**

   $$Global\_intensity = \sqrt{I_a^2 + I_b^2 + I_c^2}$$

   (similar a la intensidad global del artículo 2).

4. **Features 3 a 8:**
   Las 6 variables de fase restantes (`Va, Vb, Vc, Ia, Ib, Ic`) para darle riqueza multidimensional al Transformer, igual que el artículo 2 usa las submediciones (`Sub_metering_1..4`).

Nota: Todas estas variables se calculan a partir del dataset original `detect_dataset.csv` y se normalizan con `MinMaxScaler` antes de crear las secuencias temporales.

[13]:
```
from sklearn.preprocessing import MinMaxScaler

# ============================================
# 1. LIMPIEZA INICIAL
# ============================================
df_clean = df.drop(columns=['Unnamed: 7', 'Unnamed: 8'])

# ============================================
# 2. CREACIÓN DE FEATURES Y TARGET (CON RMS)
# ============================================

# --- Target: Potencia Activa Global ---
df_clean['Global_active_power'] = (df_clean['Va'] * np.abs(df_clean['Ia'])) + \
                                  (df_clean['Vb'] * np.abs(df_clean['Ib'])) + \
                                  (df_clean['Vc'] * np.abs(df_clean['Ic']))

# --- Feature 1: Voltaje RMS (en lugar de promedio) ---
df_clean['Voltage'] = np.sqrt((df_clean['Va']**2 + df_clean['Vb']**2 +␣
 ↪df_clean['Vc']**2) / 3)

# --- Feature 2: Intensidad Global ---
df_clean['Global_intensity'] = np.sqrt(df_clean['Ia']**2 + df_clean['Ib']**2 +␣
 ↪df_clean['Ic']**2)

# --- Features ---
feature_cols = ['Voltage', 'Global_intensity', 'Va', 'Vb', 'Vc', 'Ia', 'Ib',␣
 ↪'Ic']
target_col = 'Global_active_power'

print("Variables creadas correctamente (usando RMS para Voltage).")
print(f"Features (8): {feature_cols}")
print(f"Target: {target_col}")

# ============================================
# 3. VISUALIZACIÓN DEL NUEVO DATAFRAME (SIN NORMALIZAR)
# ============================================
print("\n" + "="*70)
print(" MUESTRA DEL DATASET CON VOLTAJE RMS (SIN NORMALIZAR)")
print("="*70)
```

```
print("Primeras 10 filas:")
print(df_clean[feature_cols + [target_col]].head(10).to_string())

# ============================================
# 4. ESTADÍSTICAS DESCRIPTIVAS
# ============================================
print("\n" + "="*70)
print(" ESTADÍSTICAS DE LAS NUEVAS VARIABLES GLOBALES (con RMS)")
print("="*70)
print(df_clean[['Global_active_power', 'Voltage', 'Global_intensity']].
 ↪describe())

# ============================================
# 5. NORMALIZACIÓN MIN-MAX
# ============================================
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()

X_scaled = scaler_X.fit_transform(df_clean[feature_cols])
y_scaled = scaler_y.fit_transform(df_clean[[target_col]]).ravel()

df_scaled = pd.DataFrame(X_scaled, columns=feature_cols)
df_scaled[target_col] = y_scaled

# ============================================
# 6. VISUALIZACIÓN DEL DATAFRAME NORMALIZADO
# ============================================
print("\n" + "="*70)
print(" MUESTRA DEL DATASET NORMALIZADO (Min-Max)")
print("="*70)
print("Primeras 10 filas:")
print(df_scaled.head(10).to_string())
print("\nEstadísticas después de normalización:")
print(df_scaled.describe().round(4))

# ============================================
# 7. CREACIÓN DE SECUENCIAS
# ============================================
WINDOW = 23

def create_sequences(data, feature_cols, target_col, window=23):
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[feature_cols].iloc[i:i+window].values)
        y.append(data[target_col].iloc[i+window])
    return np.array(X), np.array(y)
```

```python
X, y = create_sequences(df_scaled, feature_cols, target_col, WINDOW)

print(f"\nDimensiones de X (muestras, timesteps, features): {X.shape}")
print(f"Dimensiones de y: {y.shape}")

# ============================================
# 8. DIVISIÓN TRAIN/TEST
# ============================================
train_size = int(len(X) * 0.86)
X_train, X_test = X[:train_size], X[train_size:]
y_train, y_test = y[:train_size], y[train_size:]

print(f"\nTrain: X_train {X_train.shape}, y_train {y_train.shape}")
print(f"Test:  X_test {X_test.shape}, y_test {y_test.shape}")

# ============================================
# 9. GUARDAR PREPROCESADO
# ============================================
np.savez('/content/ds3_preprocessed_rms.npz',
         X_train=X_train, y_train=y_train,
         X_test=X_test, y_test=y_test,
         scaler_X=scaler_X, scaler_y=scaler_y,
         feature_cols=feature_cols, target_col=target_col)

print("\n Datos preprocesados guardados en 'ds3_preprocessed_rms.npz'")
```

```
Variables creadas correctamente (usando RMS para Voltage).
Features (8): ['Voltage', 'Global_intensity', 'Va', 'Vb', 'Vc', 'Ia', 'Ib',
'Ic']
Target: Global_active_power

======================================================================
 MUESTRA DEL DATASET CON VOLTAJE RMS (SIN NORMALIZAR)
======================================================================
Primeras 10 filas:
    Voltage  Global_intensity        Va        Vb        Vc          Ia
Ib          Ic  Global_active_power
0  0.518013        234.836467  0.054490 -0.659921  0.605431 -170.472196
9.219613  161.252583           100.832116
1  0.476950        168.674838  0.102000 -0.628612  0.526202 -122.235754
6.168667  116.067087            69.665037
2  0.447876        124.898298  0.141026 -0.605277  0.464251  -90.161474
3.813632   86.347841            50.493866
3  0.441962        111.345171  0.156272 -0.602235  0.445963  -79.904916
2.398803   77.506112            45.607142
4  0.428719         89.932639  0.180451 -0.591501  0.411050  -63.885255
0.590667   63.294587            37.196045
```

```
5  0.425896         79.849734  0.193414 -0.590695  0.397281  -55.954681
-1.001882   56.956562            32.858394
6  0.418138         65.896452  0.212393 -0.584136  0.371743  -45.248446
-2.586980   47.835426            25.881780
7  0.422194         70.213312  0.216396 -0.590123  0.373727  -47.845420
-3.428094   51.273513            27.492850
8  0.418765         64.653817  0.229748 -0.587588  0.357840  -43.294259
-4.511300   47.805558            24.402721
9  0.420979         65.625246  0.235733 -0.591320  0.355587  -43.474722
-5.388233   48.862955            24.437276

======================================================================
 ESTADÍSTICAS DE LAS NUEVAS VARIABLES GLOBALES (con RMS)
======================================================================
       Global_active_power       Voltage  Global_intensity
count         12001.000000  12001.000000      12001.000000
mean              0.033563      0.313271        426.786659
std              67.598372      0.158854        424.675327
min            -257.721035      0.010398         29.589630
25%             -24.093856      0.188510         77.688544
50%              -0.546396      0.418536        114.076117
75%              22.125544      0.428546        877.326026
max             263.517419      0.518013       1105.899391

======================================================================
 MUESTRA DEL DATASET NORMALIZADO (Min-Max)
======================================================================
Primeras 10 filas:
    Voltage  Global_intensity        Va        Vb        Vc        Ia        Ib
Ic  Global_active_power
0  1.000000          0.190695  0.548701  0.000000  0.997697  0.403028  0.508126
0.585337            0.687887
1  0.919107          0.129224  0.587308  0.024312  0.932806  0.430292  0.506422
0.560017            0.628093
2  0.861830          0.088551  0.619020  0.042432  0.882066  0.448420  0.505107
0.543364            0.591313
3  0.850180          0.075959  0.631410  0.044794  0.867087  0.454217  0.504316
0.538410            0.581937
4  0.824091          0.056065  0.651058  0.053129  0.838493  0.463271  0.503306
0.530447            0.565801
5  0.818530          0.046697  0.661591  0.053755  0.827216  0.467754  0.502417
0.526895            0.557479
6  0.803247          0.033733  0.677014  0.058849  0.806299  0.473805  0.501532
0.521784            0.544094
7  0.811238          0.037743  0.680267  0.054200  0.807924  0.472337  0.501062
0.523711            0.547185
8  0.804483          0.032578  0.691117  0.056168  0.794911  0.474909  0.500457
0.521768            0.541257
```

```
9  0.808845          0.033481  0.695980  0.053270  0.793066  0.474807  0.499967
0.522360             0.541323

Estadísticas después de normalización:
          Voltage  Global_intensity          Va          Vb          Vc  \
count  12001.0000        12001.0000  12001.0000  12001.0000  12001.0000
mean       0.5967            0.3690      0.5130      0.5004      0.5059
std        0.3129            0.3946      0.2813      0.2777      0.2861
min        0.0000            0.0000      0.0000      0.0000      0.0000
25%        0.3509            0.0447      0.3113      0.2688      0.2734
50%        0.8040            0.0785      0.5064      0.5069      0.5087
75%        0.8237            0.7876      0.7361      0.7055      0.7391
max        1.0000            1.0000      1.0000      1.0000      1.0000

                Ia          Ib          Ic  Global_active_power
count  12001.0000  12001.0000  12001.0000           12001.0000
mean       0.5032      0.4881      0.5075               0.4945
std        0.2132      0.1997      0.1693               0.1297
min        0.0000      0.0000      0.0000               0.0000
25%        0.4630      0.4743      0.4644               0.4482
50%        0.4975      0.5056      0.4948               0.4934
75%        0.5298      0.5419      0.5203               0.5369
max        1.0000      1.0000      1.0000               1.0000

Dimensiones de X (muestras, timesteps, features): (11978, 23, 8)
Dimensiones de y: (11978,)

Train: X_train (10301, 23, 8), y_train (10301,)
Test:  X_test (1677, 23, 8), y_test (1677,)

 Datos preprocesados guardados en 'ds3_preprocessed_rms.npz'
```

0.3 Configuración de Parámetros para Validación Cruzada (Basada en el Artículo 2)

Para garantizar la reproducibilidad de la metodología propuesta en el **artículo 2** (Zhang et al., "Power Consumption Predicting and Anomaly Detection Based on Transformer and K-Means") sobre el dataset **DS3** (Kaggle), se han extraído y fijado los siguientes parámetros, tal como se describen explícitamente en la sección **4.3 Model Development** y **5. EXPERIMENT RESULTS** del artículo original.

0.3.1 1. Preprocesamiento y Partición de Datos

- **Ventana deslizante (Sliding Window):** 23 horas de entrada para predecir la hora 24.
- **Partición Train/Test:**

- **Entrenamiento (Train):** 86% de los datos (equivalente a 1.240 días en el artículo original).

- **Prueba (Test):** 14% de los datos (equivalente a 200 días en el artículo original).

- **Normalización:** Se aplica `Min-Max Scaling` a todas las características (features) y a la variable objetivo (target) para estabilizar el entrenamiento y acelerar la convergencia del modelo.

0.3.2 2. Variables Adaptadas para DS3

El dataset original del artículo 2 contiene 8 variables: `Global_active_power` (target), `Global_reactive_power`, `Voltage`, `Global_intensity`, `Sub_metering_1`, `Sub_metering_2`, `Sub_metering_3`, `Sub_metering_4`.

Para adaptar DS3 a esta estructura, se calcularon las siguientes variables a partir de las mediciones de fase (`Va, Vb, Vc, Ia, Ib, Ic`):

Variable en DS3	Cálculo	Equivalente en artículo 2
`Global_active_power` (Target)	`|Va·Ia| + |Vb·Ib| + |Vc·Ic|`	`Global_active_power`
`Voltage` (Feature 1)	$\sqrt{((Va^2 + Vb^2 + Vc^2) / 3)}$ (RMS)	`Voltage`
`Global_intensity` (Feature 2)	$\sqrt{(Ia^2 + Ib^2 + Ic^2)}$	`Global_intensity`
`Va, Vb, Vc, Ia, Ib, Ic` (Features 3–8)	Variables originales de fase	`Sub_metering_1..4` (features auxiliares)

> **Nota:** Se utiliza **voltaje RMS** en lugar del promedio aritmético, ya que el promedio de las tres fases en sistemas balanceados tiende a cero, mientras que el RMS es la magnitud físicamente correcta y equivalente a la tensión eficaz del sistema.

0.3.3 3. Modelos Implementados

Siguiendo el artículo 2, se entrenan y comparan los siguientes modelos:

Modelo	Descripción	Propósito
Transformer	2 capas, 4 cabezas de atención, 8 features de entrada, 23 timesteps	Predicción de potencia
Transformer + K-Means	Transformer + optimización con K-Means (k=10)	Predicción optimizada

Modelo	Descripción	Propósito
LSTM	2 capas LSTM (32 unidades) con Dropout(0.2)	Referencia de comparación
K-Means puro	Clustering (k=10) sobre datos aplanados	Detección de anomalías por distancia al centroide

---

0.3.4 4. Configuración de Entrenamiento

Parámetro	Valor
Función de Pérdida	`MSE` (Mean Squared Error)
Optimizador	`Adam`
Épocas (Transformer)	`300` (con EarlyStopping paciencia=20)
Épocas (LSTM)	`50`
Tamaño de Lote (Batch Size)	`200` muestras (Transformer) / `64` (LSTM)
Número de Clústeres (K-Means)	`10` (probados: 2, 5, 8, 10, 11, 15)

0.3.5 5. Detección de Anomalías

El método de detección de anomalías se basa en el **score** descrito en las ecuaciones (10) y (11) del artículo:

$$\text{score}_t = \frac{|\text{predicted}_t - \text{test}_t|}{\text{avg}_{i \in T}(|\text{predicted}_i - \text{test}_i|)}$$

$$\text{score}_t^{\text{norm}} = \frac{\text{score}_t - \min(\text{score})}{\max(\text{score}) - \min(\text{score})}$$

- **Umbral:** `0.45` (establecido experimentalmente en el artículo).
- **Puntos anómalos:** aquellos con `score_norm > 0.45`.

Inserción de Anomalías Artificiales Para evaluar la detección, se siguen las instrucciones de la sección 5.2 del artículo:

- En el conjunto de prueba, se selecciona aleatoriamente un valor por cada bloque de 24 horas y se **duplica** (multiplicar por 2).
- Esto genera aproximadamente `(n_test / 24)` anomalías artificiales.
- Se comparan las detecciones de los tres métodos contra estas anomalías insertadas.

0.3.6 6. Métricas de Evaluación

Métrica	Descripción
RMSE	Root Mean Squared Error (para predicción)
Precisión	Proporción de anomalías detectadas correctamente sobre el total de detecciones
Recall	Proporción de anomalías reales detectadas
F1-Score	Media armónica de precisión y recall

0.3.7 7. Tabla de Resultados Esperados (similar a Tabla 2 del artículo)

Method	Precisión	Recall	F1	RMSE (predicción)
K-Means puro	~0.82	~0.28	~0.42	—
LSTM	~0.74	~0.60	~0.66	~0.91
Transformer + K-Means (nuestro método)	~0.80	~0.66	~0.72	~0.74

0.3.8 8. Artefactos Generados

Archivo	Descripción
`ds3_preprocessed_rms.npz`	Datos preprocesados (X_train, X_test, y_train, y_test, scalers)
`transformer_model.h5`	Modelo Transformer entrenado
`kmeans_model.pkl`	K-Means para optimización de predicción
`lstm_model.h5`	Modelo LSTM entrenado
`kmeans_anomaly_model.pkl`	K-Means para detección de anomalías
`scaler_X.pkl`, `scaler_y.pkl`	Objetos MinMaxScaler
`training_history.npz`	Historial de pérdidas del Transformer
`validation_results.csv`	Métricas finales (RMSE, precisión, recall, F1)
`validation_results.png`	Gráficos de comparación de predicciones y detección de anomalías

---

[5]:
```
!pip install tensorflow scikit-learn joblib
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-
packages (2.20.0)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-
packages (1.6.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages
(1.5.3)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-
```

```
packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (26.2)
Requirement already satisfied: protobuf>=5.28.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (2.2.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (1.81.0)
Requirement already satisfied: tensorboard~=2.20.0 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-
packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in
/usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-
packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in
/usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow)
(0.47.0)
```

```
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages
(from keras>=3.10.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages
(from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages
(from keras>=3.10.0->tensorflow) (0.19.1)
Requirement already satisfied: charset_normalizer<4,>=2 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
(3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-
packages (from requests<3,>=2.21.0->tensorflow) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
(2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
(2026.5.20)
Requirement already satisfied: markdown>=2.6.8 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow)
(3.10.2)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages
(from tensorboard~=2.20.0->tensorflow) (11.3.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow)
(0.7.2)
Requirement already satisfied: werkzeug>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow)
(3.1.8)
Requirement already satisfied: markupsafe>=2.1.1 in
/usr/local/lib/python3.12/dist-packages (from
werkzeug>=1.0.1->tensorboard~=2.20.0->tensorflow) (3.0.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow)
(4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tensorflow)
(2.20.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-
packages (from markdown-it-py>=2.2.0->rich->keras>=3.10.0->tensorflow) (0.1.2)
```

[16]:
```python
# ================================================================
# FASE 3a: ENTRENAMIENTO DEL TRANSFORMER + K-MEANS (OPTIMIZACIÓN)
# Basado en el artículo 2 y en el código original del autor
# ================================================================

import os
os.chdir("/content")  # Asegurar directorio correcto
```

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model
from sklearn.cluster import KMeans
import joblib

BASE_DIR = "/content"

# ------------------------------------------------------------------
# 1. Cargar datos procesados
# ------------------------------------------------------------------
print("Cargando datos procesados...")
data = np.load(f"{BASE_DIR}/ds3_preprocessed_rms.npz", allow_pickle=True)
X_train = data["X_train"]
y_train = data["y_train"]
X_test = data["X_test"]
y_test = data["y_test"]

print(f"X_train: {X_train.shape}, y_train: {y_train.shape}")
print(f"X_test: {X_test.shape}, y_test: {y_test.shape}")

# ------------------------------------------------------------------
# 2. Entrenar K-Means sobre y_train (k=10)
# ------------------------------------------------------------------
print("\nEntrenando K-Means sobre y_train (k=10)...")
kmeans_opt = KMeans(n_clusters=10, random_state=42, n_init=10)
kmeans_opt.fit(y_train.reshape(-1, 1))
centroids = kmeans_opt.cluster_centers_.flatten()
print(f"Centroides: {centroids}")

joblib.dump(kmeans_opt, f"{BASE_DIR}/kmeans_model.pkl")
print("K-Means guardado en 'kmeans_model.pkl'")

# ------------------------------------------------------------------
# 3. Definir Transformer
# ------------------------------------------------------------------
def transformer_encoder(inputs, head_size=8, num_heads=4, ff_dim=32, dropout=0.
 ↪1):
    attention = layers.MultiHeadAttention(num_heads=num_heads,␣
 ↪key_dim=head_size)
    x = attention(inputs, inputs)
    x = layers.Dropout(dropout)(x)
    x = layers.LayerNormalization(epsilon=1e-6)(x + inputs)
    x_ff = layers.Dense(ff_dim, activation="relu")(x)
    x_ff = layers.Dense(inputs.shape[-1])(x_ff)
    x_ff = layers.Dropout(dropout)(x_ff)
```

```
    return layers.LayerNormalization(epsilon=1e-6)(x + x_ff)

def build_transformer(input_shape, num_layers=2, head_size=8, num_heads=4,␣
 ↪ff_dim=32, dropout=0.1):
    inputs = layers.Input(shape=input_shape)
    x = inputs
    for _ in range(num_layers):
        x = transformer_encoder(x, head_size, num_heads, ff_dim, dropout)
    x = layers.Lambda(lambda t: t[:, -1, :])(x)
    outputs = layers.Dense(1)(x)
    return Model(inputs, outputs)

INPUT_SHAPE = (23, 8)
transformer = build_transformer(INPUT_SHAPE, num_layers=2, head_size=2,␣
 ↪num_heads=4, ff_dim=32, dropout=0.1)
transformer.summary()

# ---------------------------------------------------------------------
# 4. Compilar
# ---------------------------------------------------------------------
transformer.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                    loss='mse', metrics=['mae'])

# ---------------------------------------------------------------------
# 5. Entrenar
# ---------------------------------------------------------------------
print("\nIniciando entrenamiento del Transformer...")
history = transformer.fit(
    X_train, y_train,
    validation_split=0.1,
    epochs=300,
    batch_size=200,
    verbose=1,
    callbacks=[
        tf.keras.callbacks.ModelCheckpoint(f"{BASE_DIR}/best_transformer.
 ↪keras", save_best_only=True, monitor='val_loss')
    ]
)

transformer.save(f"{BASE_DIR}/transformer_final.keras")
print("Transformer guardado.")

joblib.dump(history.history, f"{BASE_DIR}/training_history.pkl")
print("Historial guardado.")

print("\nEntrenamiento del Transformer completado.")
```

```
Cargando datos procesados…
X_train: (10301, 23, 8), y_train: (10301,)
X_test: (1677, 23, 8), y_test: (1677,)

Entrenando K-Means sobre y_train (k=10)…
Centroides: [0.36025018 0.66665284 0.54608692 0.18519321 0.952794   0.44980085
 0.78311995 0.04510666 0.2752222  0.49860481]
K-Means guardado en 'kmeans_model.pkl'
```

Model: "functional_4"

Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, 23, 8)	0	-
multi_head_attenti… (MultiHeadAttentio…	(None, 23, 8)	288	input_layer_4[0]… input_layer_4[0]…
dropout_25 (Dropout)	(None, 23, 8)	0	multi_head_atten…
add_12 (Add)	(None, 23, 8)	0	dropout_25[0][0], input_layer_4[0]…
layer_normalizatio… (LayerNormalizatio…	(None, 23, 8)	16	add_12[0][0]
dense_19 (Dense)	(None, 23, 32)	288	layer_normalizat…
dense_20 (Dense)	(None, 23, 8)	264	dense_19[0][0]
dropout_26 (Dropout)	(None, 23, 8)	0	dense_20[0][0]
add_13 (Add)	(None, 23, 8)	0	layer_normalizat… dropout_26[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 23, 8)	16	add_13[0][0]
multi_head_attenti… (MultiHeadAttentio…	(None, 23, 8)	288	layer_normalizat… layer_normalizat…
dropout_28 (Dropout)	(None, 23, 8)	0	multi_head_atten…

add_14 (Add)	(None, 23, 8)	0	dropout_28[0][0], layer_normalizat…
layer_normalizatio… (LayerNormalizatio…	(None, 23, 8)	16	add_14[0][0]
dense_21 (Dense)	(None, 23, 32)	288	layer_normalizat…
dense_22 (Dense)	(None, 23, 8)	264	dense_21[0][0]
dropout_29 (Dropout)	(None, 23, 8)	0	dense_22[0][0]
add_15 (Add)	(None, 23, 8)	0	layer_normalizat… dropout_29[0][0]
layer_normalizatio… (LayerNormalizatio…	(None, 23, 8)	16	add_15[0][0]
lambda (Lambda)	(None, 8)	0	layer_normalizat…
dense_23 (Dense)	(None, 1)	9	lambda[0][0]

Total params: 1,753 (6.85 KB)

Trainable params: 1,753 (6.85 KB)

Non-trainable params: 0 (0.00 B)

```
Iniciando entrenamiento del Transformer…
Epoch 1/300
47/47              21s 181ms/step -
loss: 0.3692 - mae: 0.4346 - val_loss: 0.0077 - val_mae: 0.0696
Epoch 2/300
47/47              0s 8ms/step - loss:
0.0311 - mae: 0.1230 - val_loss: 0.0040 - val_mae: 0.0474
Epoch 3/300
47/47              0s 8ms/step - loss:
0.0256 - mae: 0.1105 - val_loss: 0.0037 - val_mae: 0.0452
Epoch 4/300
47/47              0s 8ms/step - loss:
0.0236 - mae: 0.1063 - val_loss: 0.0036 - val_mae: 0.0448
Epoch 5/300
47/47              0s 8ms/step - loss:
```

```
0.0220 - mae: 0.1033 - val_loss: 0.0035 - val_mae: 0.0439
Epoch 6/300
47/47            0s 6ms/step - loss:
0.0215 - mae: 0.1021 - val_loss: 0.0035 - val_mae: 0.0444
Epoch 7/300
47/47            0s 7ms/step - loss:
0.0207 - mae: 0.1004 - val_loss: 0.0035 - val_mae: 0.0443
Epoch 8/300
47/47            0s 9ms/step - loss:
0.0201 - mae: 0.0998 - val_loss: 0.0034 - val_mae: 0.0434
Epoch 9/300
47/47            0s 9ms/step - loss:
0.0196 - mae: 0.0984 - val_loss: 0.0033 - val_mae: 0.0436
Epoch 10/300
47/47            0s 9ms/step - loss:
0.0194 - mae: 0.0984 - val_loss: 0.0031 - val_mae: 0.0423
Epoch 11/300
47/47            0s 9ms/step - loss:
0.0186 - mae: 0.0964 - val_loss: 0.0030 - val_mae: 0.0413
Epoch 12/300
47/47            0s 8ms/step - loss:
0.0181 - mae: 0.0953 - val_loss: 0.0029 - val_mae: 0.0409
Epoch 13/300
47/47            0s 9ms/step - loss:
0.0177 - mae: 0.0949 - val_loss: 0.0027 - val_mae: 0.0389
Epoch 14/300
47/47            0s 9ms/step - loss:
0.0169 - mae: 0.0929 - val_loss: 0.0025 - val_mae: 0.0374
Epoch 15/300
47/47            0s 9ms/step - loss:
0.0163 - mae: 0.0914 - val_loss: 0.0023 - val_mae: 0.0341
Epoch 16/300
47/47            0s 9ms/step - loss:
0.0157 - mae: 0.0898 - val_loss: 0.0021 - val_mae: 0.0332
Epoch 17/300
47/47            0s 6ms/step - loss:
0.0152 - mae: 0.0878 - val_loss: 0.0022 - val_mae: 0.0345
Epoch 18/300
47/47            0s 9ms/step - loss:
0.0147 - mae: 0.0860 - val_loss: 0.0020 - val_mae: 0.0332
Epoch 19/300
47/47            0s 8ms/step - loss:
0.0144 - mae: 0.0841 - val_loss: 0.0019 - val_mae: 0.0307
Epoch 20/300
47/47            0s 10ms/step -
loss: 0.0140 - mae: 0.0824 - val_loss: 0.0018 - val_mae: 0.0304
Epoch 21/300
47/47            1s 12ms/step -
```

```
loss: 0.0135 - mae: 0.0813 - val_loss: 0.0017 - val_mae: 0.0308
Epoch 22/300
47/47            1s 12ms/step -
loss: 0.0133 - mae: 0.0798 - val_loss: 0.0016 - val_mae: 0.0304
Epoch 23/300
47/47            1s 12ms/step -
loss: 0.0127 - mae: 0.0782 - val_loss: 0.0015 - val_mae: 0.0288
Epoch 24/300
47/47            0s 9ms/step - loss:
0.0124 - mae: 0.0769 - val_loss: 0.0015 - val_mae: 0.0291
Epoch 25/300
47/47            0s 9ms/step - loss:
0.0121 - mae: 0.0761 - val_loss: 0.0012 - val_mae: 0.0266
Epoch 26/300
47/47            0s 9ms/step - loss:
0.0119 - mae: 0.0750 - val_loss: 0.0012 - val_mae: 0.0265
Epoch 27/300
47/47            0s 9ms/step - loss:
0.0113 - mae: 0.0738 - val_loss: 0.0010 - val_mae: 0.0253
Epoch 28/300
47/47            0s 6ms/step - loss:
0.0111 - mae: 0.0725 - val_loss: 0.0011 - val_mae: 0.0256
Epoch 29/300
47/47            0s 9ms/step - loss:
0.0107 - mae: 0.0715 - val_loss: 9.0621e-04 - val_mae: 0.0238
Epoch 30/300
47/47            0s 6ms/step - loss:
0.0102 - mae: 0.0701 - val_loss: 9.7620e-04 - val_mae: 0.0252
Epoch 31/300
47/47            0s 8ms/step - loss:
0.0096 - mae: 0.0680 - val_loss: 8.3044e-04 - val_mae: 0.0230
Epoch 32/300
47/47            0s 6ms/step - loss:
0.0091 - mae: 0.0660 - val_loss: 8.6447e-04 - val_mae: 0.0242
Epoch 33/300
47/47            0s 8ms/step - loss:
0.0085 - mae: 0.0642 - val_loss: 6.9459e-04 - val_mae: 0.0211
Epoch 34/300
47/47            0s 6ms/step - loss:
0.0080 - mae: 0.0625 - val_loss: 7.7156e-04 - val_mae: 0.0232
Epoch 35/300
47/47            0s 6ms/step - loss:
0.0075 - mae: 0.0607 - val_loss: 8.0897e-04 - val_mae: 0.0238
Epoch 36/300
47/47            0s 6ms/step - loss:
0.0070 - mae: 0.0596 - val_loss: 8.0899e-04 - val_mae: 0.0237
Epoch 37/300
47/47            0s 6ms/step - loss:
```

```
0.0062 - mae: 0.0560 - val_loss: 7.5925e-04 - val_mae: 0.0229
Epoch 38/300
47/47            0s 6ms/step - loss:
0.0059 - mae: 0.0554 - val_loss: 8.1794e-04 - val_mae: 0.0226
Epoch 39/300
47/47            0s 7ms/step - loss:
0.0057 - mae: 0.0536 - val_loss: 8.5137e-04 - val_mae: 0.0252
Epoch 40/300
47/47            0s 6ms/step - loss:
0.0051 - mae: 0.0515 - val_loss: 9.4630e-04 - val_mae: 0.0258
Epoch 41/300
47/47            0s 7ms/step - loss:
0.0049 - mae: 0.0504 - val_loss: 9.6413e-04 - val_mae: 0.0259
Epoch 42/300
47/47            0s 6ms/step - loss:
0.0047 - mae: 0.0489 - val_loss: 8.9704e-04 - val_mae: 0.0257
Epoch 43/300
47/47            0s 6ms/step - loss:
0.0044 - mae: 0.0476 - val_loss: 9.0441e-04 - val_mae: 0.0261
Epoch 44/300
47/47            0s 6ms/step - loss:
0.0041 - mae: 0.0463 - val_loss: 0.0010 - val_mae: 0.0265
Epoch 45/300
47/47            0s 6ms/step - loss:
0.0040 - mae: 0.0456 - val_loss: 8.9331e-04 - val_mae: 0.0249
Epoch 46/300
47/47            0s 6ms/step - loss:
0.0037 - mae: 0.0438 - val_loss: 9.6781e-04 - val_mae: 0.0258
Epoch 47/300
47/47            0s 6ms/step - loss:
0.0036 - mae: 0.0435 - val_loss: 9.5811e-04 - val_mae: 0.0255
Epoch 48/300
47/47            0s 6ms/step - loss:
0.0035 - mae: 0.0427 - val_loss: 0.0010 - val_mae: 0.0260
Epoch 49/300
47/47            0s 6ms/step - loss:
0.0035 - mae: 0.0425 - val_loss: 0.0010 - val_mae: 0.0268
Epoch 50/300
47/47            0s 6ms/step - loss:
0.0032 - mae: 0.0410 - val_loss: 0.0010 - val_mae: 0.0271
Epoch 51/300
47/47            0s 6ms/step - loss:
0.0030 - mae: 0.0400 - val_loss: 8.6161e-04 - val_mae: 0.0246
Epoch 52/300
47/47            0s 6ms/step - loss:
0.0032 - mae: 0.0405 - val_loss: 9.5050e-04 - val_mae: 0.0258
Epoch 53/300
47/47            0s 7ms/step - loss:
```

```
0.0031 - mae: 0.0397 - val_loss: 9.9283e-04 - val_mae: 0.0264
Epoch 54/300
47/47            1s 9ms/step - loss:
0.0030 - mae: 0.0397 - val_loss: 9.3898e-04 - val_mae: 0.0260
Epoch 55/300
47/47            0s 8ms/step - loss:
0.0029 - mae: 0.0386 - val_loss: 9.2877e-04 - val_mae: 0.0255
Epoch 56/300
47/47            1s 9ms/step - loss:
0.0029 - mae: 0.0385 - val_loss: 9.5434e-04 - val_mae: 0.0264
Epoch 57/300
47/47            0s 6ms/step - loss:
0.0029 - mae: 0.0384 - val_loss: 9.6647e-04 - val_mae: 0.0261
Epoch 58/300
47/47            0s 7ms/step - loss:
0.0026 - mae: 0.0371 - val_loss: 9.3535e-04 - val_mae: 0.0253
Epoch 59/300
47/47            0s 6ms/step - loss:
0.0026 - mae: 0.0368 - val_loss: 0.0010 - val_mae: 0.0269
Epoch 60/300
47/47            0s 6ms/step - loss:
0.0026 - mae: 0.0365 - val_loss: 8.0042e-04 - val_mae: 0.0237
Epoch 61/300
47/47            0s 6ms/step - loss:
0.0024 - mae: 0.0352 - val_loss: 8.1076e-04 - val_mae: 0.0244
Epoch 62/300
47/47            0s 6ms/step - loss:
0.0025 - mae: 0.0358 - val_loss: 9.0626e-04 - val_mae: 0.0251
Epoch 63/300
47/47            0s 6ms/step - loss:
0.0024 - mae: 0.0354 - val_loss: 9.5161e-04 - val_mae: 0.0261
Epoch 64/300
47/47            0s 6ms/step - loss:
0.0024 - mae: 0.0352 - val_loss: 9.0676e-04 - val_mae: 0.0247
Epoch 65/300
47/47            0s 6ms/step - loss:
0.0024 - mae: 0.0350 - val_loss: 8.8197e-04 - val_mae: 0.0257
Epoch 66/300
47/47            0s 6ms/step - loss:
0.0023 - mae: 0.0344 - val_loss: 9.3615e-04 - val_mae: 0.0253
Epoch 67/300
47/47            0s 6ms/step - loss:
0.0023 - mae: 0.0340 - val_loss: 8.8194e-04 - val_mae: 0.0245
Epoch 68/300
47/47            0s 6ms/step - loss:
0.0023 - mae: 0.0340 - val_loss: 9.5698e-04 - val_mae: 0.0255
Epoch 69/300
47/47            0s 6ms/step - loss:
```

```
0.0021 - mae: 0.0334 - val_loss: 9.5919e-04 - val_mae: 0.0249
Epoch 70/300
47/47            0s 6ms/step - loss:
0.0022 - mae: 0.0336 - val_loss: 8.0636e-04 - val_mae: 0.0236
Epoch 71/300
47/47            0s 7ms/step - loss:
0.0023 - mae: 0.0334 - val_loss: 9.2645e-04 - val_mae: 0.0254
Epoch 72/300
47/47            0s 6ms/step - loss:
0.0022 - mae: 0.0332 - val_loss: 9.2196e-04 - val_mae: 0.0253
Epoch 73/300
47/47            0s 7ms/step - loss:
0.0022 - mae: 0.0337 - val_loss: 8.2624e-04 - val_mae: 0.0238
Epoch 74/300
47/47            0s 6ms/step - loss:
0.0021 - mae: 0.0327 - val_loss: 9.1116e-04 - val_mae: 0.0255
Epoch 75/300
47/47            0s 6ms/step - loss:
0.0021 - mae: 0.0328 - val_loss: 7.3789e-04 - val_mae: 0.0226
Epoch 76/300
47/47            0s 6ms/step - loss:
0.0020 - mae: 0.0319 - val_loss: 8.5486e-04 - val_mae: 0.0238
Epoch 77/300
47/47            0s 6ms/step - loss:
0.0020 - mae: 0.0321 - val_loss: 9.3245e-04 - val_mae: 0.0250
Epoch 78/300
47/47            0s 6ms/step - loss:
0.0019 - mae: 0.0311 - val_loss: 8.4145e-04 - val_mae: 0.0248
Epoch 79/300
47/47            0s 6ms/step - loss:
0.0019 - mae: 0.0314 - val_loss: 8.5753e-04 - val_mae: 0.0247
Epoch 80/300
47/47            0s 6ms/step - loss:
0.0019 - mae: 0.0312 - val_loss: 9.0600e-04 - val_mae: 0.0249
Epoch 81/300
47/47            0s 6ms/step - loss:
0.0019 - mae: 0.0310 - val_loss: 8.5031e-04 - val_mae: 0.0246
Epoch 82/300
47/47            0s 6ms/step - loss:
0.0018 - mae: 0.0304 - val_loss: 8.3167e-04 - val_mae: 0.0233
Epoch 83/300
47/47            0s 6ms/step - loss:
0.0018 - mae: 0.0299 - val_loss: 7.8771e-04 - val_mae: 0.0235
Epoch 84/300
47/47            0s 6ms/step - loss:
0.0018 - mae: 0.0304 - val_loss: 8.7744e-04 - val_mae: 0.0251
Epoch 85/300
47/47            0s 6ms/step - loss:
```

```
0.0018 - mae: 0.0303 - val_loss: 0.0011 - val_mae: 0.0287
Epoch 86/300
47/47            0s 6ms/step - loss:
0.0017 - mae: 0.0299 - val_loss: 8.5899e-04 - val_mae: 0.0249
Epoch 87/300
47/47            0s 6ms/step - loss:
0.0018 - mae: 0.0305 - val_loss: 7.7808e-04 - val_mae: 0.0238
Epoch 88/300
47/47            0s 8ms/step - loss:
0.0017 - mae: 0.0298 - val_loss: 0.0011 - val_mae: 0.0283
Epoch 89/300
47/47            1s 8ms/step - loss:
0.0016 - mae: 0.0289 - val_loss: 8.5731e-04 - val_mae: 0.0240
Epoch 90/300
47/47            0s 8ms/step - loss:
0.0016 - mae: 0.0283 - val_loss: 7.4668e-04 - val_mae: 0.0224
Epoch 91/300
47/47            0s 8ms/step - loss:
0.0017 - mae: 0.0289 - val_loss: 7.4458e-04 - val_mae: 0.0234
Epoch 92/300
47/47            0s 8ms/step - loss:
0.0016 - mae: 0.0285 - val_loss: 0.0010 - val_mae: 0.0272
Epoch 93/300
47/47            0s 6ms/step - loss:
0.0016 - mae: 0.0285 - val_loss: 7.7841e-04 - val_mae: 0.0232
Epoch 94/300
47/47            0s 6ms/step - loss:
0.0016 - mae: 0.0283 - val_loss: 0.0010 - val_mae: 0.0272
Epoch 95/300
47/47            0s 7ms/step - loss:
0.0015 - mae: 0.0280 - val_loss: 8.0966e-04 - val_mae: 0.0242
Epoch 96/300
47/47            0s 6ms/step - loss:
0.0015 - mae: 0.0280 - val_loss: 7.2602e-04 - val_mae: 0.0228
Epoch 97/300
47/47            0s 6ms/step - loss:
0.0015 - mae: 0.0285 - val_loss: 7.9927e-04 - val_mae: 0.0237
Epoch 98/300
47/47            0s 6ms/step - loss:
0.0015 - mae: 0.0276 - val_loss: 7.3637e-04 - val_mae: 0.0228
Epoch 99/300
47/47            1s 6ms/step - loss:
0.0014 - mae: 0.0273 - val_loss: 7.8857e-04 - val_mae: 0.0236
Epoch 100/300
47/47            0s 6ms/step - loss:
0.0014 - mae: 0.0271 - val_loss: 8.3229e-04 - val_mae: 0.0247
Epoch 101/300
47/47            0s 6ms/step - loss:
```

```
0.0014 - mae: 0.0271 - val_loss: 7.5977e-04 - val_mae: 0.0241
Epoch 102/300
47/47            1s 6ms/step - loss:
0.0014 - mae: 0.0267 - val_loss: 7.8311e-04 - val_mae: 0.0237
Epoch 103/300
47/47            0s 6ms/step - loss:
0.0014 - mae: 0.0268 - val_loss: 8.6618e-04 - val_mae: 0.0254
Epoch 104/300
47/47            0s 6ms/step - loss:
0.0014 - mae: 0.0266 - val_loss: 8.2311e-04 - val_mae: 0.0252
Epoch 105/300
47/47            1s 7ms/step - loss:
0.0014 - mae: 0.0268 - val_loss: 7.4847e-04 - val_mae: 0.0231
Epoch 106/300
47/47            1s 12ms/step -
loss: 0.0014 - mae: 0.0270 - val_loss: 7.6798e-04 - val_mae: 0.0228
Epoch 107/300
47/47            0s 6ms/step - loss:
0.0014 - mae: 0.0264 - val_loss: 8.1490e-04 - val_mae: 0.0242
Epoch 108/300
47/47            1s 14ms/step -
loss: 0.0013 - mae: 0.0261 - val_loss: 7.2408e-04 - val_mae: 0.0225
Epoch 109/300
47/47            0s 6ms/step - loss:
0.0013 - mae: 0.0254 - val_loss: 7.6135e-04 - val_mae: 0.0228
Epoch 110/300
47/47            0s 6ms/step - loss:
0.0013 - mae: 0.0257 - val_loss: 7.3831e-04 - val_mae: 0.0226
Epoch 111/300
47/47            1s 6ms/step - loss:
0.0013 - mae: 0.0260 - val_loss: 7.5696e-04 - val_mae: 0.0233
Epoch 112/300
47/47            1s 6ms/step - loss:
0.0012 - mae: 0.0251 - val_loss: 7.3686e-04 - val_mae: 0.0230
Epoch 113/300
47/47            1s 7ms/step - loss:
0.0012 - mae: 0.0252 - val_loss: 8.4681e-04 - val_mae: 0.0243
Epoch 114/300
47/47            0s 6ms/step - loss:
0.0013 - mae: 0.0255 - val_loss: 7.3227e-04 - val_mae: 0.0221
Epoch 115/300
47/47            0s 6ms/step - loss:
0.0012 - mae: 0.0246 - val_loss: 8.2899e-04 - val_mae: 0.0243
Epoch 116/300
47/47            0s 9ms/step - loss:
0.0012 - mae: 0.0250 - val_loss: 7.2166e-04 - val_mae: 0.0217
Epoch 117/300
47/47            0s 8ms/step - loss:
```

```
0.0012 - mae: 0.0246 - val_loss: 7.3573e-04 - val_mae: 0.0231
Epoch 118/300
47/47            0s 8ms/step - loss:
0.0012 - mae: 0.0245 - val_loss: 7.4162e-04 - val_mae: 0.0228
Epoch 119/300
47/47            1s 12ms/step -
loss: 0.0012 - mae: 0.0246 - val_loss: 6.6136e-04 - val_mae: 0.0214
Epoch 120/300
47/47            0s 7ms/step - loss:
0.0012 - mae: 0.0242 - val_loss: 7.6540e-04 - val_mae: 0.0234
Epoch 121/300
47/47            1s 6ms/step - loss:
0.0011 - mae: 0.0240 - val_loss: 6.9798e-04 - val_mae: 0.0224
Epoch 122/300
47/47            0s 6ms/step - loss:
0.0012 - mae: 0.0245 - val_loss: 7.4141e-04 - val_mae: 0.0230
Epoch 123/300
47/47            0s 6ms/step - loss:
0.0011 - mae: 0.0239 - val_loss: 7.3018e-04 - val_mae: 0.0217
Epoch 124/300
47/47            0s 6ms/step - loss:
0.0011 - mae: 0.0239 - val_loss: 6.8321e-04 - val_mae: 0.0226
Epoch 125/300
47/47            0s 6ms/step - loss:
0.0011 - mae: 0.0237 - val_loss: 8.1065e-04 - val_mae: 0.0230
Epoch 126/300
47/47            0s 6ms/step - loss:
0.0011 - mae: 0.0235 - val_loss: 8.2249e-04 - val_mae: 0.0239
Epoch 127/300
47/47            0s 6ms/step - loss:
0.0011 - mae: 0.0235 - val_loss: 7.8601e-04 - val_mae: 0.0222
Epoch 128/300
47/47            0s 9ms/step - loss:
0.0011 - mae: 0.0237 - val_loss: 6.5210e-04 - val_mae: 0.0214
Epoch 129/300
47/47            0s 6ms/step - loss:
0.0010 - mae: 0.0230 - val_loss: 9.4031e-04 - val_mae: 0.0258
Epoch 130/300
47/47            0s 6ms/step - loss:
0.0010 - mae: 0.0231 - val_loss: 7.2626e-04 - val_mae: 0.0224
Epoch 131/300
47/47            0s 6ms/step - loss:
0.0010 - mae: 0.0231 - val_loss: 7.6063e-04 - val_mae: 0.0227
Epoch 132/300
47/47            0s 9ms/step - loss:
9.7364e-04 - mae: 0.0225 - val_loss: 6.3299e-04 - val_mae: 0.0212
Epoch 133/300
47/47            0s 6ms/step - loss:
```

```
9.9411e-04 - mae: 0.0228 - val_loss: 7.0809e-04 - val_mae: 0.0215
Epoch 134/300
47/47             0s 6ms/step - loss:
0.0010 - mae: 0.0227 - val_loss: 7.7487e-04 - val_mae: 0.0229
Epoch 135/300
47/47             0s 6ms/step - loss:
9.5487e-04 - mae: 0.0221 - val_loss: 8.3121e-04 - val_mae: 0.0238
Epoch 136/300
47/47             0s 9ms/step - loss:
9.7656e-04 - mae: 0.0224 - val_loss: 6.2834e-04 - val_mae: 0.0212
Epoch 137/300
47/47             0s 6ms/step - loss:
9.5147e-04 - mae: 0.0220 - val_loss: 7.3771e-04 - val_mae: 0.0211
Epoch 138/300
47/47             1s 6ms/step - loss:
9.4929e-04 - mae: 0.0220 - val_loss: 6.7632e-04 - val_mae: 0.0196
Epoch 139/300
47/47             0s 6ms/step - loss:
9.4669e-04 - mae: 0.0221 - val_loss: 6.6355e-04 - val_mae: 0.0218
Epoch 140/300
47/47             0s 6ms/step - loss:
9.0524e-04 - mae: 0.0216 - val_loss: 7.0849e-04 - val_mae: 0.0230
Epoch 141/300
47/47             0s 6ms/step - loss:
9.2442e-04 - mae: 0.0217 - val_loss: 6.6986e-04 - val_mae: 0.0222
Epoch 142/300
47/47             0s 6ms/step - loss:
8.5742e-04 - mae: 0.0211 - val_loss: 7.7902e-04 - val_mae: 0.0235
Epoch 143/300
47/47             1s 16ms/step -
loss: 9.3074e-04 - mae: 0.0219 - val_loss: 5.9744e-04 - val_mae: 0.0205
Epoch 144/300
47/47             1s 11ms/step -
loss: 9.0207e-04 - mae: 0.0213 - val_loss: 6.8836e-04 - val_mae: 0.0225
Epoch 145/300
47/47             0s 9ms/step - loss:
8.5895e-04 - mae: 0.0211 - val_loss: 6.1534e-04 - val_mae: 0.0199
Epoch 146/300
47/47             0s 9ms/step - loss:
8.5164e-04 - mae: 0.0209 - val_loss: 6.7283e-04 - val_mae: 0.0219
Epoch 147/300
47/47             1s 12ms/step -
loss: 8.9062e-04 - mae: 0.0213 - val_loss: 5.9723e-04 - val_mae: 0.0202
Epoch 148/300
47/47             0s 9ms/step - loss:
8.8836e-04 - mae: 0.0213 - val_loss: 6.8560e-04 - val_mae: 0.0221
Epoch 149/300
47/47             0s 9ms/step - loss:
```

```
8.6097e-04 - mae: 0.0210 - val_loss: 5.4809e-04 - val_mae: 0.0195
Epoch 150/300
47/47             0s 6ms/step - loss:
8.2767e-04 - mae: 0.0205 - val_loss: 7.4116e-04 - val_mae: 0.0222
Epoch 151/300
47/47             0s 6ms/step - loss:
8.2381e-04 - mae: 0.0206 - val_loss: 7.1975e-04 - val_mae: 0.0228
Epoch 152/300
47/47             0s 6ms/step - loss:
8.0264e-04 - mae: 0.0203 - val_loss: 6.4074e-04 - val_mae: 0.0220
Epoch 153/300
47/47             0s 6ms/step - loss:
7.9737e-04 - mae: 0.0203 - val_loss: 6.9485e-04 - val_mae: 0.0221
Epoch 154/300
47/47             0s 6ms/step - loss:
8.0012e-04 - mae: 0.0204 - val_loss: 6.6752e-04 - val_mae: 0.0224
Epoch 155/300
47/47             0s 6ms/step - loss:
8.1224e-04 - mae: 0.0204 - val_loss: 5.6475e-04 - val_mae: 0.0194
Epoch 156/300
47/47             0s 6ms/step - loss:
8.2661e-04 - mae: 0.0205 - val_loss: 5.6071e-04 - val_mae: 0.0193
Epoch 157/300
47/47             0s 6ms/step - loss:
7.8937e-04 - mae: 0.0200 - val_loss: 7.0582e-04 - val_mae: 0.0220
Epoch 158/300
47/47             0s 6ms/step - loss:
7.5455e-04 - mae: 0.0197 - val_loss: 7.7994e-04 - val_mae: 0.0240
Epoch 159/300
47/47             0s 6ms/step - loss:
7.5438e-04 - mae: 0.0199 - val_loss: 6.9369e-04 - val_mae: 0.0209
Epoch 160/300
47/47             0s 6ms/step - loss:
7.4280e-04 - mae: 0.0196 - val_loss: 6.8372e-04 - val_mae: 0.0220
Epoch 161/300
47/47             0s 6ms/step - loss:
7.4634e-04 - mae: 0.0193 - val_loss: 6.7204e-04 - val_mae: 0.0216
Epoch 162/300
47/47             0s 6ms/step - loss:
7.2175e-04 - mae: 0.0193 - val_loss: 6.2454e-04 - val_mae: 0.0204
Epoch 163/300
47/47             0s 6ms/step - loss:
7.6452e-04 - mae: 0.0195 - val_loss: 6.4336e-04 - val_mae: 0.0216
Epoch 164/300
47/47             0s 6ms/step - loss:
7.4761e-04 - mae: 0.0197 - val_loss: 5.9587e-04 - val_mae: 0.0209
Epoch 165/300
47/47             0s 6ms/step - loss:
```

```
7.1951e-04 - mae: 0.0195 - val_loss: 6.6218e-04 - val_mae: 0.0218
Epoch 166/300
47/47             0s 6ms/step - loss:
7.2228e-04 - mae: 0.0193 - val_loss: 5.6125e-04 - val_mae: 0.0195
Epoch 167/300
47/47             0s 6ms/step - loss:
7.0918e-04 - mae: 0.0191 - val_loss: 6.5494e-04 - val_mae: 0.0207
Epoch 168/300
47/47             0s 6ms/step - loss:
7.3446e-04 - mae: 0.0194 - val_loss: 5.6161e-04 - val_mae: 0.0198
Epoch 169/300
47/47             0s 6ms/step - loss:
6.8311e-04 - mae: 0.0188 - val_loss: 5.8004e-04 - val_mae: 0.0205
Epoch 170/300
47/47             0s 6ms/step - loss:
6.9420e-04 - mae: 0.0187 - val_loss: 5.4945e-04 - val_mae: 0.0193
Epoch 171/300
47/47             0s 6ms/step - loss:
6.5642e-04 - mae: 0.0188 - val_loss: 6.4379e-04 - val_mae: 0.0212
Epoch 172/300
47/47             0s 6ms/step - loss:
6.7879e-04 - mae: 0.0189 - val_loss: 7.3606e-04 - val_mae: 0.0219
Epoch 173/300
47/47             0s 9ms/step - loss:
6.6604e-04 - mae: 0.0187 - val_loss: 5.4236e-04 - val_mae: 0.0194
Epoch 174/300
47/47             0s 6ms/step - loss:
6.8057e-04 - mae: 0.0186 - val_loss: 5.4647e-04 - val_mae: 0.0191
Epoch 175/300
47/47             0s 9ms/step - loss:
6.4936e-04 - mae: 0.0186 - val_loss: 4.8368e-04 - val_mae: 0.0183
Epoch 176/300
47/47             0s 6ms/step - loss:
6.6877e-04 - mae: 0.0188 - val_loss: 6.3357e-04 - val_mae: 0.0206
Epoch 177/300
47/47             0s 6ms/step - loss:
6.5755e-04 - mae: 0.0186 - val_loss: 7.1027e-04 - val_mae: 0.0225
Epoch 178/300
47/47             1s 7ms/step - loss:
6.8072e-04 - mae: 0.0189 - val_loss: 6.3726e-04 - val_mae: 0.0217
Epoch 179/300
47/47             0s 9ms/step - loss:
6.3715e-04 - mae: 0.0183 - val_loss: 6.2462e-04 - val_mae: 0.0214
Epoch 180/300
47/47             0s 10ms/step -
loss: 6.0229e-04 - mae: 0.0179 - val_loss: 6.1435e-04 - val_mae: 0.0205
Epoch 181/300
47/47             0s 8ms/step - loss:
```

```
6.1510e-04 - mae: 0.0178 - val_loss: 5.9058e-04 - val_mae: 0.0198
Epoch 182/300
47/47             0s 9ms/step - loss:
6.0935e-04 - mae: 0.0179 - val_loss: 6.9701e-04 - val_mae: 0.0221
Epoch 183/300
47/47             0s 8ms/step - loss:
6.3250e-04 - mae: 0.0180 - val_loss: 5.8746e-04 - val_mae: 0.0196
Epoch 184/300
47/47             0s 7ms/step - loss:
5.8308e-04 - mae: 0.0176 - val_loss: 5.8152e-04 - val_mae: 0.0204
Epoch 185/300
47/47             0s 6ms/step - loss:
6.0425e-04 - mae: 0.0179 - val_loss: 5.9398e-04 - val_mae: 0.0197
Epoch 186/300
47/47             0s 6ms/step - loss:
6.0448e-04 - mae: 0.0179 - val_loss: 5.5196e-04 - val_mae: 0.0197
Epoch 187/300
47/47             0s 6ms/step - loss:
5.8181e-04 - mae: 0.0173 - val_loss: 6.0370e-04 - val_mae: 0.0213
Epoch 188/300
47/47             0s 7ms/step - loss:
5.8132e-04 - mae: 0.0175 - val_loss: 5.8559e-04 - val_mae: 0.0217
Epoch 189/300
47/47             0s 6ms/step - loss:
5.6954e-04 - mae: 0.0173 - val_loss: 5.5414e-04 - val_mae: 0.0200
Epoch 190/300
47/47             1s 7ms/step - loss:
5.8178e-04 - mae: 0.0175 - val_loss: 5.3453e-04 - val_mae: 0.0188
Epoch 191/300
47/47             0s 6ms/step - loss:
5.6125e-04 - mae: 0.0172 - val_loss: 6.0923e-04 - val_mae: 0.0212
Epoch 192/300
47/47             0s 6ms/step - loss:
5.6129e-04 - mae: 0.0172 - val_loss: 6.0679e-04 - val_mae: 0.0211
Epoch 193/300
47/47             0s 7ms/step - loss:
5.6531e-04 - mae: 0.0172 - val_loss: 5.2852e-04 - val_mae: 0.0180
Epoch 194/300
47/47             0s 6ms/step - loss:
5.6733e-04 - mae: 0.0173 - val_loss: 6.0046e-04 - val_mae: 0.0210
Epoch 195/300
47/47             0s 6ms/step - loss:
5.4211e-04 - mae: 0.0170 - val_loss: 5.5319e-04 - val_mae: 0.0198
Epoch 196/300
47/47             0s 7ms/step - loss:
5.5400e-04 - mae: 0.0172 - val_loss: 5.6604e-04 - val_mae: 0.0197
Epoch 197/300
47/47             0s 7ms/step - loss:
```

```
5.3603e-04 - mae: 0.0169 - val_loss: 5.5269e-04 - val_mae: 0.0197
Epoch 198/300
47/47             0s 6ms/step - loss:
5.5712e-04 - mae: 0.0171 - val_loss: 6.4105e-04 - val_mae: 0.0192
Epoch 199/300
47/47             0s 9ms/step - loss:
5.4191e-04 - mae: 0.0168 - val_loss: 4.6918e-04 - val_mae: 0.0177
Epoch 200/300
47/47             0s 6ms/step - loss:
5.3800e-04 - mae: 0.0170 - val_loss: 6.0367e-04 - val_mae: 0.0199
Epoch 201/300
47/47             0s 7ms/step - loss:
5.0141e-04 - mae: 0.0164 - val_loss: 5.9467e-04 - val_mae: 0.0206
Epoch 202/300
47/47             0s 6ms/step - loss:
5.2588e-04 - mae: 0.0165 - val_loss: 5.6079e-04 - val_mae: 0.0204
Epoch 203/300
47/47             0s 7ms/step - loss:
5.0128e-04 - mae: 0.0163 - val_loss: 5.8406e-04 - val_mae: 0.0191
Epoch 204/300
47/47             0s 6ms/step - loss:
5.5382e-04 - mae: 0.0171 - val_loss: 5.3294e-04 - val_mae: 0.0186
Epoch 205/300
47/47             0s 6ms/step - loss:
5.2053e-04 - mae: 0.0166 - val_loss: 5.3891e-04 - val_mae: 0.0196
Epoch 206/300
47/47             1s 6ms/step - loss:
5.2164e-04 - mae: 0.0166 - val_loss: 5.2373e-04 - val_mae: 0.0189
Epoch 207/300
47/47             0s 6ms/step - loss:
5.3266e-04 - mae: 0.0165 - val_loss: 4.7553e-04 - val_mae: 0.0179
Epoch 208/300
47/47             1s 9ms/step - loss:
4.8927e-04 - mae: 0.0161 - val_loss: 4.6187e-04 - val_mae: 0.0171
Epoch 209/300
47/47             0s 7ms/step - loss:
5.2654e-04 - mae: 0.0166 - val_loss: 5.6454e-04 - val_mae: 0.0207
Epoch 210/300
47/47             0s 8ms/step - loss:
4.9277e-04 - mae: 0.0161 - val_loss: 4.9843e-04 - val_mae: 0.0172
Epoch 211/300
47/47             0s 9ms/step - loss:
4.8170e-04 - mae: 0.0159 - val_loss: 5.0518e-04 - val_mae: 0.0188
Epoch 212/300
47/47             0s 9ms/step - loss:
4.8029e-04 - mae: 0.0161 - val_loss: 6.5009e-04 - val_mae: 0.0219
Epoch 213/300
47/47             0s 9ms/step - loss:
```

```
4.9378e-04 - mae: 0.0159 - val_loss: 5.8476e-04 - val_mae: 0.0195
Epoch 214/300
47/47             0s 9ms/step - loss:
4.8253e-04 - mae: 0.0160 - val_loss: 4.9290e-04 - val_mae: 0.0183
Epoch 215/300
47/47             0s 8ms/step - loss:
4.8156e-04 - mae: 0.0159 - val_loss: 6.2400e-04 - val_mae: 0.0189
Epoch 216/300
47/47             0s 6ms/step - loss:
4.6566e-04 - mae: 0.0158 - val_loss: 5.7320e-04 - val_mae: 0.0198
Epoch 217/300
47/47             0s 7ms/step - loss:
5.0729e-04 - mae: 0.0164 - val_loss: 5.0256e-04 - val_mae: 0.0178
Epoch 218/300
47/47             0s 6ms/step - loss:
4.8423e-04 - mae: 0.0158 - val_loss: 5.7850e-04 - val_mae: 0.0202
Epoch 219/300
47/47             1s 6ms/step - loss:
4.6756e-04 - mae: 0.0157 - val_loss: 5.0864e-04 - val_mae: 0.0178
Epoch 220/300
47/47             0s 7ms/step - loss:
5.1448e-04 - mae: 0.0163 - val_loss: 5.7684e-04 - val_mae: 0.0205
Epoch 221/300
47/47             0s 6ms/step - loss:
4.6657e-04 - mae: 0.0156 - val_loss: 6.1655e-04 - val_mae: 0.0210
Epoch 222/300
47/47             0s 6ms/step - loss:
4.6884e-04 - mae: 0.0158 - val_loss: 6.2273e-04 - val_mae: 0.0206
Epoch 223/300
47/47             1s 7ms/step - loss:
4.6432e-04 - mae: 0.0158 - val_loss: 5.1795e-04 - val_mae: 0.0191
Epoch 224/300
47/47             0s 7ms/step - loss:
4.4853e-04 - mae: 0.0155 - val_loss: 5.4202e-04 - val_mae: 0.0188
Epoch 225/300
47/47             0s 7ms/step - loss:
4.5547e-04 - mae: 0.0155 - val_loss: 5.3928e-04 - val_mae: 0.0190
Epoch 226/300
47/47             0s 6ms/step - loss:
4.6972e-04 - mae: 0.0159 - val_loss: 5.7172e-04 - val_mae: 0.0199
Epoch 227/300
47/47             0s 6ms/step - loss:
4.3698e-04 - mae: 0.0153 - val_loss: 5.6732e-04 - val_mae: 0.0201
Epoch 228/300
47/47             0s 7ms/step - loss:
4.2049e-04 - mae: 0.0148 - val_loss: 5.5494e-04 - val_mae: 0.0200
Epoch 229/300
47/47             0s 6ms/step - loss:
```

```
4.2721e-04 - mae: 0.0152 - val_loss: 4.6651e-04 - val_mae: 0.0182
Epoch 230/300
47/47             0s 6ms/step - loss:
4.2107e-04 - mae: 0.0149 - val_loss: 5.0632e-04 - val_mae: 0.0191
Epoch 231/300
47/47             0s 7ms/step - loss:
4.3330e-04 - mae: 0.0152 - val_loss: 5.9963e-04 - val_mae: 0.0201
Epoch 232/300
47/47             0s 7ms/step - loss:
4.5035e-04 - mae: 0.0156 - val_loss: 5.1881e-04 - val_mae: 0.0186
Epoch 233/300
47/47             0s 6ms/step - loss:
4.3342e-04 - mae: 0.0152 - val_loss: 5.6294e-04 - val_mae: 0.0200
Epoch 234/300
47/47             0s 7ms/step - loss:
4.2014e-04 - mae: 0.0151 - val_loss: 4.7556e-04 - val_mae: 0.0183
Epoch 235/300
47/47             0s 6ms/step - loss:
4.5904e-04 - mae: 0.0154 - val_loss: 5.2735e-04 - val_mae: 0.0187
Epoch 236/300
47/47             0s 6ms/step - loss:
4.7217e-04 - mae: 0.0157 - val_loss: 5.0850e-04 - val_mae: 0.0190
Epoch 237/300
47/47             1s 6ms/step - loss:
4.1333e-04 - mae: 0.0147 - val_loss: 5.1433e-04 - val_mae: 0.0197
Epoch 238/300
47/47             1s 6ms/step - loss:
4.4731e-04 - mae: 0.0153 - val_loss: 4.9016e-04 - val_mae: 0.0193
Epoch 239/300
47/47             0s 7ms/step - loss:
4.2431e-04 - mae: 0.0151 - val_loss: 6.0076e-04 - val_mae: 0.0204
Epoch 240/300
47/47             0s 6ms/step - loss:
4.4969e-04 - mae: 0.0154 - val_loss: 5.2064e-04 - val_mae: 0.0180
Epoch 241/300
47/47             0s 6ms/step - loss:
4.3975e-04 - mae: 0.0152 - val_loss: 5.1004e-04 - val_mae: 0.0198
Epoch 242/300
47/47             0s 8ms/step - loss:
4.1668e-04 - mae: 0.0149 - val_loss: 4.9397e-04 - val_mae: 0.0181
Epoch 243/300
47/47             0s 8ms/step - loss:
4.2415e-04 - mae: 0.0148 - val_loss: 4.9818e-04 - val_mae: 0.0184
Epoch 244/300
47/47             1s 9ms/step - loss:
4.1407e-04 - mae: 0.0145 - val_loss: 4.9905e-04 - val_mae: 0.0184
Epoch 245/300
47/47             1s 9ms/step - loss:
```

```
4.0958e-04 - mae: 0.0148 - val_loss: 5.3852e-04 - val_mae: 0.0194
Epoch 246/300
47/47             0s 8ms/step - loss:
4.0534e-04 - mae: 0.0148 - val_loss: 5.5379e-04 - val_mae: 0.0185
Epoch 247/300
47/47             0s 7ms/step - loss:
4.0750e-04 - mae: 0.0149 - val_loss: 5.9550e-04 - val_mae: 0.0215
Epoch 248/300
47/47             0s 6ms/step - loss:
4.2843e-04 - mae: 0.0152 - val_loss: 5.4654e-04 - val_mae: 0.0194
Epoch 249/300
47/47             0s 9ms/step - loss:
4.1868e-04 - mae: 0.0149 - val_loss: 4.5536e-04 - val_mae: 0.0173
Epoch 250/300
47/47             0s 6ms/step - loss:
4.2037e-04 - mae: 0.0148 - val_loss: 5.0143e-04 - val_mae: 0.0183
Epoch 251/300
47/47             0s 6ms/step - loss:
3.8436e-04 - mae: 0.0144 - val_loss: 5.2783e-04 - val_mae: 0.0195
Epoch 252/300
47/47             1s 6ms/step - loss:
4.0785e-04 - mae: 0.0147 - val_loss: 5.2907e-04 - val_mae: 0.0185
Epoch 253/300
47/47             0s 6ms/step - loss:
3.8266e-04 - mae: 0.0144 - val_loss: 4.5949e-04 - val_mae: 0.0173
Epoch 254/300
47/47             0s 7ms/step - loss:
3.7664e-04 - mae: 0.0142 - val_loss: 4.7993e-04 - val_mae: 0.0179
Epoch 255/300
47/47             1s 6ms/step - loss:
3.9747e-04 - mae: 0.0146 - val_loss: 4.8378e-04 - val_mae: 0.0189
Epoch 256/300
47/47             0s 7ms/step - loss:
3.9098e-04 - mae: 0.0144 - val_loss: 4.9453e-04 - val_mae: 0.0182
Epoch 257/300
47/47             1s 6ms/step - loss:
3.8807e-04 - mae: 0.0144 - val_loss: 4.6044e-04 - val_mae: 0.0177
Epoch 258/300
47/47             0s 6ms/step - loss:
3.7269e-04 - mae: 0.0140 - val_loss: 4.6924e-04 - val_mae: 0.0177
Epoch 259/300
47/47             0s 6ms/step - loss:
3.6887e-04 - mae: 0.0140 - val_loss: 5.8986e-04 - val_mae: 0.0215
Epoch 260/300
47/47             0s 6ms/step - loss:
3.8317e-04 - mae: 0.0143 - val_loss: 4.6310e-04 - val_mae: 0.0176
Epoch 261/300
47/47             1s 7ms/step - loss:
```

```
3.7554e-04 - mae: 0.0141 - val_loss: 5.2739e-04 - val_mae: 0.0187
Epoch 262/300
47/47             0s 7ms/step - loss:
3.5925e-04 - mae: 0.0138 - val_loss: 5.0456e-04 - val_mae: 0.0173
Epoch 263/300
47/47             0s 6ms/step - loss:
3.8197e-04 - mae: 0.0142 - val_loss: 5.9472e-04 - val_mae: 0.0195
Epoch 264/300
47/47             0s 6ms/step - loss:
3.5882e-04 - mae: 0.0139 - val_loss: 4.6507e-04 - val_mae: 0.0171
Epoch 265/300
47/47             0s 6ms/step - loss:
3.5310e-04 - mae: 0.0137 - val_loss: 4.7586e-04 - val_mae: 0.0188
Epoch 266/300
47/47             0s 6ms/step - loss:
3.5343e-04 - mae: 0.0139 - val_loss: 4.5704e-04 - val_mae: 0.0178
Epoch 267/300
47/47             0s 10ms/step -
loss: 3.9404e-04 - mae: 0.0144 - val_loss: 4.4155e-04 - val_mae: 0.0174
Epoch 268/300
47/47             0s 6ms/step - loss:
3.7138e-04 - mae: 0.0140 - val_loss: 4.9269e-04 - val_mae: 0.0181
Epoch 269/300
47/47             1s 9ms/step - loss:
3.7927e-04 - mae: 0.0144 - val_loss: 4.3226e-04 - val_mae: 0.0171
Epoch 270/300
47/47             0s 9ms/step - loss:
3.6976e-04 - mae: 0.0139 - val_loss: 4.2277e-04 - val_mae: 0.0170
Epoch 271/300
47/47             1s 12ms/step -
loss: 3.5837e-04 - mae: 0.0139 - val_loss: 4.1032e-04 - val_mae: 0.0165
Epoch 272/300
47/47             0s 9ms/step - loss:
3.5313e-04 - mae: 0.0136 - val_loss: 5.2926e-04 - val_mae: 0.0191
Epoch 273/300
47/47             1s 10ms/step -
loss: 3.7220e-04 - mae: 0.0140 - val_loss: 4.1366e-04 - val_mae: 0.0167
Epoch 274/300
47/47             0s 10ms/step -
loss: 3.6692e-04 - mae: 0.0140 - val_loss: 4.5129e-04 - val_mae: 0.0183
Epoch 275/300
47/47             0s 7ms/step - loss:
3.7008e-04 - mae: 0.0140 - val_loss: 5.1217e-04 - val_mae: 0.0195
Epoch 276/300
47/47             1s 9ms/step - loss:
3.6020e-04 - mae: 0.0138 - val_loss: 3.8239e-04 - val_mae: 0.0159
Epoch 277/300
47/47             0s 7ms/step - loss:
```

```
3.5991e-04 - mae: 0.0138 - val_loss: 5.2457e-04 - val_mae: 0.0203
Epoch 278/300
47/47             0s 6ms/step - loss:
3.5913e-04 - mae: 0.0137 - val_loss: 4.3084e-04 - val_mae: 0.0167
Epoch 279/300
47/47             0s 6ms/step - loss:
3.5332e-04 - mae: 0.0137 - val_loss: 4.2576e-04 - val_mae: 0.0174
Epoch 280/300
47/47             1s 7ms/step - loss:
3.5399e-04 - mae: 0.0136 - val_loss: 4.5258e-04 - val_mae: 0.0177
Epoch 281/300
47/47             0s 6ms/step - loss:
3.7709e-04 - mae: 0.0141 - val_loss: 4.0198e-04 - val_mae: 0.0161
Epoch 282/300
47/47             1s 7ms/step - loss:
3.5925e-04 - mae: 0.0140 - val_loss: 4.3281e-04 - val_mae: 0.0168
Epoch 283/300
47/47             0s 6ms/step - loss:
3.3207e-04 - mae: 0.0133 - val_loss: 4.5703e-04 - val_mae: 0.0175
Epoch 284/300
47/47             1s 7ms/step - loss:
3.6901e-04 - mae: 0.0141 - val_loss: 4.4093e-04 - val_mae: 0.0165
Epoch 285/300
47/47             0s 6ms/step - loss:
3.4306e-04 - mae: 0.0135 - val_loss: 4.6170e-04 - val_mae: 0.0181
Epoch 286/300
47/47             0s 6ms/step - loss:
3.6622e-04 - mae: 0.0136 - val_loss: 6.0696e-04 - val_mae: 0.0204
Epoch 287/300
47/47             0s 7ms/step - loss:
3.4907e-04 - mae: 0.0136 - val_loss: 4.7109e-04 - val_mae: 0.0178
Epoch 288/300
47/47             0s 6ms/step - loss:
3.4427e-04 - mae: 0.0133 - val_loss: 4.4890e-04 - val_mae: 0.0171
Epoch 289/300
47/47             0s 8ms/step - loss:
3.4428e-04 - mae: 0.0136 - val_loss: 4.6365e-04 - val_mae: 0.0180
Epoch 290/300
47/47             1s 16ms/step -
loss: 3.3810e-04 - mae: 0.0136 - val_loss: 4.4180e-04 - val_mae: 0.0182
Epoch 291/300
47/47             1s 11ms/step -
loss: 3.3599e-04 - mae: 0.0132 - val_loss: 4.2615e-04 - val_mae: 0.0170
Epoch 292/300
47/47             1s 16ms/step -
loss: 3.5656e-04 - mae: 0.0137 - val_loss: 4.6212e-04 - val_mae: 0.0177
Epoch 293/300
47/47             1s 12ms/step -
```

```
loss: 3.3235e-04 - mae: 0.0133 - val_loss: 4.5888e-04 - val_mae: 0.0180
Epoch 294/300
47/47             0s 8ms/step - loss:
3.2749e-04 - mae: 0.0130 - val_loss: 4.7758e-04 - val_mae: 0.0177
Epoch 295/300
47/47             1s 10ms/step -
loss: 3.3631e-04 - mae: 0.0133 - val_loss: 4.9127e-04 - val_mae: 0.0181
Epoch 296/300
47/47             0s 9ms/step - loss:
3.2690e-04 - mae: 0.0132 - val_loss: 4.0754e-04 - val_mae: 0.0166
Epoch 297/300
47/47             0s 9ms/step - loss:
3.3845e-04 - mae: 0.0131 - val_loss: 3.9421e-04 - val_mae: 0.0161
Epoch 298/300
47/47             0s 9ms/step - loss:
3.3258e-04 - mae: 0.0132 - val_loss: 4.0183e-04 - val_mae: 0.0162
Epoch 299/300
47/47             0s 6ms/step - loss:
3.3004e-04 - mae: 0.0133 - val_loss: 4.4506e-04 - val_mae: 0.0171
Epoch 300/300
47/47             1s 6ms/step - loss:
3.4771e-04 - mae: 0.0136 - val_loss: 4.2863e-04 - val_mae: 0.0169
Transformer guardado.
Historial guardado.

Entrenamiento del Transformer completado.
```

[17]:

```python
# ====================================================================
# ENTRENAMIENTO DE MODELOS BASELINE (LSTM y K-MEANS)
# Basado en el artículo 2 y en el código original del autor
# ====================================================================

import os
os.chdir("/content")

import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from sklearn.cluster import KMeans
import joblib

BASE_DIR = "/content"

# --------------------------------------------------------------------
# 1. Cargar datos
# --------------------------------------------------------------------
```

```
print("Cargando datos procesados...")
data = np.load(f"{BASE_DIR}/ds3_preprocessed_rms.npz", allow_pickle=True)
X_train = data["X_train"]
y_train = data["y_train"].flatten()
X_test = data["X_test"]
y_test = data["y_test"].flatten()

print(f"X_train: {X_train.shape}, y_train: {y_train.shape}")

# ------------------------------------------------------------------
# 2. LSTM
# ------------------------------------------------------------------
LSTM_PATH = f"{BASE_DIR}/lstm_model.keras"
if os.path.exists(LSTM_PATH):
    print(f"LSTM ya existe en {LSTM_PATH}. Se omite entrenamiento.")
else:
    print("\nEntrenando LSTM...")
    lstm = Sequential([
        LSTM(64, return_sequences=True, input_shape=(23, 8)),
        Dropout(0.2),
        LSTM(32, return_sequences=False),
        Dropout(0.2),
        Dense(1)
    ])
    lstm.compile(optimizer='adam', loss='mse', metrics=['mae'])
    early_stop = tf.keras.callbacks.EarlyStopping(patience=10,␣
 ↪restore_best_weights=True)
    lstm.fit(X_train, y_train, epochs=100, batch_size=200, validation_split=0.1,
             callbacks=[early_stop], verbose=1)
    lstm.save(LSTM_PATH)
    print(f"LSTM guardado en {LSTM_PATH}")

# ------------------------------------------------------------------
# 3. K-Means baseline (para detección de anomalías)
# ------------------------------------------------------------------
print("\nEntrenando K-Means baseline (k=10)...")
X_train_flat = X_train.reshape(X_train.shape[0], -1)
kmeans_base = KMeans(n_clusters=10, random_state=42, n_init=10)
kmeans_base.fit(X_train_flat)
joblib.dump(kmeans_base, f"{BASE_DIR}/kmeans_baseline.pkl")
print("K-Means baseline guardado.")

print("\nFase 3b completada.")
```

```
Cargando datos procesados…
X_train: (10301, 23, 8), y_train: (10301,)
```

```
Entrenando LSTM…
Epoch 1/100
47/47            3s 16ms/step -
loss: 0.0375 - mae: 0.1441 - val_loss: 0.0049 - val_mae: 0.0538
Epoch 2/100
47/47            0s 9ms/step - loss:
0.0222 - mae: 0.1117 - val_loss: 0.0053 - val_mae: 0.0567
Epoch 3/100
47/47            0s 9ms/step - loss:
0.0206 - mae: 0.1083 - val_loss: 0.0060 - val_mae: 0.0612
Epoch 4/100
47/47            0s 9ms/step - loss:
0.0182 - mae: 0.1014 - val_loss: 0.0048 - val_mae: 0.0534
Epoch 5/100
47/47            0s 9ms/step - loss:
0.0155 - mae: 0.0920 - val_loss: 0.0067 - val_mae: 0.0634
Epoch 6/100
47/47            0s 10ms/step -
loss: 0.0143 - mae: 0.0892 - val_loss: 0.0039 - val_mae: 0.0469
Epoch 7/100
47/47            1s 14ms/step -
loss: 0.0118 - mae: 0.0803 - val_loss: 0.0025 - val_mae: 0.0352
Epoch 8/100
47/47            1s 14ms/step -
loss: 0.0099 - mae: 0.0739 - val_loss: 0.0051 - val_mae: 0.0560
Epoch 9/100
47/47            1s 16ms/step -
loss: 0.0087 - mae: 0.0706 - val_loss: 0.0026 - val_mae: 0.0364
Epoch 10/100
47/47            1s 10ms/step -
loss: 0.0076 - mae: 0.0675 - val_loss: 0.0022 - val_mae: 0.0379
Epoch 11/100
47/47            0s 9ms/step - loss:
0.0060 - mae: 0.0600 - val_loss: 0.0028 - val_mae: 0.0427
Epoch 12/100
47/47            0s 9ms/step - loss:
0.0054 - mae: 0.0570 - val_loss: 0.0019 - val_mae: 0.0351
Epoch 13/100
47/47            0s 9ms/step - loss:
0.0051 - mae: 0.0556 - val_loss: 0.0020 - val_mae: 0.0369
Epoch 14/100
47/47            0s 9ms/step - loss:
0.0048 - mae: 0.0532 - val_loss: 0.0017 - val_mae: 0.0334
Epoch 15/100
47/47            0s 9ms/step - loss:
0.0047 - mae: 0.0536 - val_loss: 0.0032 - val_mae: 0.0464
Epoch 16/100
47/47            0s 9ms/step - loss:
```

```
0.0044 - mae: 0.0511 - val_loss: 0.0026 - val_mae: 0.0417
Epoch 17/100
47/47            0s 10ms/step -
loss: 0.0041 - mae: 0.0500 - val_loss: 0.0027 - val_mae: 0.0407
Epoch 18/100
47/47            0s 9ms/step - loss:
0.0040 - mae: 0.0488 - val_loss: 0.0024 - val_mae: 0.0389
Epoch 19/100
47/47            1s 9ms/step - loss:
0.0038 - mae: 0.0474 - val_loss: 0.0029 - val_mae: 0.0422
Epoch 20/100
47/47            0s 9ms/step - loss:
0.0037 - mae: 0.0473 - val_loss: 0.0020 - val_mae: 0.0367
Epoch 21/100
47/47            0s 9ms/step - loss:
0.0035 - mae: 0.0459 - val_loss: 0.0029 - val_mae: 0.0440
Epoch 22/100
47/47            0s 9ms/step - loss:
0.0035 - mae: 0.0459 - val_loss: 0.0016 - val_mae: 0.0351
Epoch 23/100
47/47            0s 9ms/step - loss:
0.0035 - mae: 0.0455 - val_loss: 0.0023 - val_mae: 0.0395
Epoch 24/100
47/47            0s 9ms/step - loss:
0.0031 - mae: 0.0430 - val_loss: 0.0018 - val_mae: 0.0344
Epoch 25/100
47/47            0s 9ms/step - loss:
0.0028 - mae: 0.0409 - val_loss: 0.0015 - val_mae: 0.0317
Epoch 26/100
47/47            0s 9ms/step - loss:
0.0028 - mae: 0.0405 - val_loss: 0.0024 - val_mae: 0.0410
Epoch 27/100
47/47            0s 9ms/step - loss:
0.0029 - mae: 0.0413 - val_loss: 0.0018 - val_mae: 0.0338
Epoch 28/100
47/47            0s 9ms/step - loss:
0.0025 - mae: 0.0388 - val_loss: 0.0024 - val_mae: 0.0420
Epoch 29/100
47/47            0s 9ms/step - loss:
0.0027 - mae: 0.0397 - val_loss: 0.0017 - val_mae: 0.0338
Epoch 30/100
47/47            1s 11ms/step -
loss: 0.0024 - mae: 0.0372 - val_loss: 0.0016 - val_mae: 0.0331
Epoch 31/100
47/47            1s 14ms/step -
loss: 0.0022 - mae: 0.0358 - val_loss: 0.0019 - val_mae: 0.0321
Epoch 32/100
47/47            1s 14ms/step -
```

```
loss: 0.0022 - mae: 0.0360 - val_loss: 0.0014 - val_mae: 0.0296
Epoch 33/100
47/47            1s 14ms/step -
loss: 0.0021 - mae: 0.0348 - val_loss: 0.0014 - val_mae: 0.0291
Epoch 34/100
47/47            0s 9ms/step - loss:
0.0020 - mae: 0.0342 - val_loss: 0.0018 - val_mae: 0.0326
Epoch 35/100
47/47            0s 9ms/step - loss:
0.0020 - mae: 0.0346 - val_loss: 0.0013 - val_mae: 0.0286
Epoch 36/100
47/47            0s 9ms/step - loss:
0.0019 - mae: 0.0337 - val_loss: 0.0015 - val_mae: 0.0312
Epoch 37/100
47/47            0s 9ms/step - loss:
0.0020 - mae: 0.0343 - val_loss: 0.0015 - val_mae: 0.0306
Epoch 38/100
47/47            0s 9ms/step - loss:
0.0018 - mae: 0.0322 - val_loss: 0.0013 - val_mae: 0.0287
Epoch 39/100
47/47            0s 9ms/step - loss:
0.0017 - mae: 0.0313 - val_loss: 0.0014 - val_mae: 0.0314
Epoch 40/100
47/47            0s 9ms/step - loss:
0.0017 - mae: 0.0315 - val_loss: 0.0012 - val_mae: 0.0284
Epoch 41/100
47/47            0s 10ms/step -
loss: 0.0016 - mae: 0.0305 - val_loss: 0.0010 - val_mae: 0.0254
Epoch 42/100
47/47            0s 9ms/step - loss:
0.0015 - mae: 0.0295 - val_loss: 0.0011 - val_mae: 0.0258
Epoch 43/100
47/47            0s 9ms/step - loss:
0.0016 - mae: 0.0298 - val_loss: 0.0011 - val_mae: 0.0266
Epoch 44/100
47/47            1s 10ms/step -
loss: 0.0015 - mae: 0.0294 - val_loss: 0.0014 - val_mae: 0.0297
Epoch 45/100
47/47            0s 9ms/step - loss:
0.0015 - mae: 0.0293 - val_loss: 0.0010 - val_mae: 0.0244
Epoch 46/100
47/47            0s 9ms/step - loss:
0.0014 - mae: 0.0282 - val_loss: 0.0012 - val_mae: 0.0287
Epoch 47/100
47/47            0s 9ms/step - loss:
0.0013 - mae: 0.0276 - val_loss: 0.0013 - val_mae: 0.0254
Epoch 48/100
47/47            1s 9ms/step - loss:
```

```
0.0013 - mae: 0.0273 - val_loss: 8.3266e-04 - val_mae: 0.0230
Epoch 49/100
47/47            0s 9ms/step - loss:
0.0014 - mae: 0.0280 - val_loss: 0.0014 - val_mae: 0.0302
Epoch 50/100
47/47            0s 9ms/step - loss:
0.0013 - mae: 0.0270 - val_loss: 0.0010 - val_mae: 0.0268
Epoch 51/100
47/47            0s 9ms/step - loss:
0.0012 - mae: 0.0263 - val_loss: 8.4044e-04 - val_mae: 0.0222
Epoch 52/100
47/47            0s 9ms/step - loss:
0.0011 - mae: 0.0251 - val_loss: 0.0010 - val_mae: 0.0258
Epoch 53/100
47/47            0s 9ms/step - loss:
0.0011 - mae: 0.0251 - val_loss: 6.4950e-04 - val_mae: 0.0207
Epoch 54/100
47/47            0s 9ms/step - loss:
0.0011 - mae: 0.0246 - val_loss: 0.0011 - val_mae: 0.0255
Epoch 55/100
47/47            1s 14ms/step -
loss: 0.0011 - mae: 0.0248 - val_loss: 8.5173e-04 - val_mae: 0.0228
Epoch 56/100
47/47            1s 15ms/step -
loss: 0.0010 - mae: 0.0242 - val_loss: 7.3281e-04 - val_mae: 0.0205
Epoch 57/100
47/47            0s 10ms/step -
loss: 9.5098e-04 - mae: 0.0231 - val_loss: 6.9360e-04 - val_mae: 0.0207
Epoch 58/100
47/47            0s 9ms/step - loss:
0.0010 - mae: 0.0243 - val_loss: 0.0010 - val_mae: 0.0248
Epoch 59/100
47/47            1s 10ms/step -
loss: 9.4988e-04 - mae: 0.0231 - val_loss: 7.0274e-04 - val_mae: 0.0207
Epoch 60/100
47/47            0s 9ms/step - loss:
9.6322e-04 - mae: 0.0232 - val_loss: 0.0016 - val_mae: 0.0282
Epoch 61/100
47/47            0s 9ms/step - loss:
9.6525e-04 - mae: 0.0232 - val_loss: 7.8799e-04 - val_mae: 0.0203
Epoch 62/100
47/47            0s 9ms/step - loss:
8.8407e-04 - mae: 0.0222 - val_loss: 7.6965e-04 - val_mae: 0.0214
Epoch 63/100
47/47            0s 9ms/step - loss:
9.0854e-04 - mae: 0.0225 - val_loss: 7.9627e-04 - val_mae: 0.0227
LSTM guardado en /content/lstm_model.keras
```

```
Entrenando K-Means baseline (k=10)…
K-Means baseline guardado.

Fase 3b completada.
```

1 Evaluación y Detección de Anomalías usando el modelo entrenado

El artículo reporta métricas específicas que debemos intentar reproducir. Usaremos el modelo guardado (best_transformer.keras o transformer_final.keras) y el K-Means entrenado.

[20]:
```python
# ======================================================================
# FASE 4: EVALUACIÓN Y DETECCIÓN DE ANOMALÍAS (CORREGIDO)
# ======================================================================

import os
os.chdir("/content")

import numpy as np
import tensorflow as tf
import joblib
from sklearn.metrics import accuracy_score, precision_score, recall_score,␣
 ↪f1_score, mean_squared_error
import pandas as pd
import warnings
warnings.filterwarnings('ignore')

# Habilitar deserialización de Lambdas (por la capa Lambda del Transformer)
tf.keras.config.enable_unsafe_deserialization()

BASE_DIR = "/content"

# ----------------------------------------------------------------------
# 1. Cargar datos y modelos
# ----------------------------------------------------------------------
print("Cargando datos...")
data = np.load(f"{BASE_DIR}/ds3_preprocessed_rms.npz", allow_pickle=True)
X_test = data["X_test"]
y_test_norm = data["y_test"].flatten()

# Cargar scaler_y
scaler_y = joblib.load(f"{BASE_DIR}/scaler_y.pkl")
min_active = scaler_y.data_min_[0]
max_active = scaler_y.data_max_[0]

def inverse_transform_active(norm_value):
    return norm_value * (max_active - min_active) + min_active
```

```
# Cargar Transformer
print("Cargando modelo Transformer...")
transformer = tf.keras.models.load_model(
    f"{BASE_DIR}/best_transformer.keras",
    compile=False,
    safe_mode=False
)

# Cargar K-Means (entrenado sobre y_train)
print("Cargando K-Means (optimización)...")
kmeans_opt = joblib.load(f"{BASE_DIR}/kmeans_model.pkl")

# Obtener centroides directamente (están en el espacio de y_train, 1D)
centroids = kmeans_opt.cluster_centers_.flatten()
print(f"Centroides de K-Means (en espacio de y): {centroids}")

# Cargar LSTM (opcional)
lstm_path = f"{BASE_DIR}/lstm_model.keras"
if os.path.exists(lstm_path):
    lstm = tf.keras.models.load_model(lstm_path, compile=False, safe_mode=False)
    print("Modelo LSTM cargado.")
else:
    lstm = None
    print("Modelo LSTM no encontrado. Se omite comparación.")

# ------------------------------------------------------------------
# 2. Predicción y RMSE (Transformer + K-Means)
# ------------------------------------------------------------------
print("\nGenerando predicciones con Transformer + K-Means...")
y_pred_trans = transformer.predict(X_test, verbose=0).flatten()

# Corregir con K-Means: asignar cada predicción al centroide más cercano
y_pred_corrected = np.array([
    centroids[np.argmin(np.abs(y_pred_trans[i] - centroids))]
    for i in range(len(y_pred_trans))
])

y_test_orig = inverse_transform_active(y_test_norm)
y_pred_orig = inverse_transform_active(y_pred_corrected)

rmse = np.sqrt(mean_squared_error(y_test_orig, y_pred_orig))
print(f"RMSE (Transformer + K-Means) en escala original: {rmse:.4f} kW")

if lstm is not None:
    y_pred_lstm = lstm.predict(X_test, verbose=0).flatten()
    y_pred_lstm_orig = inverse_transform_active(y_pred_lstm)
```

```
    rmse_lstm = np.sqrt(mean_squared_error(y_test_orig, y_pred_lstm_orig))
    print(f"RMSE (LSTM) en escala original: {rmse_lstm:.4f} kW")
else:
    rmse_lstm = None

# ------------------------------------------------------------------
# 3. Inyección de anomalías (1 por día, multiplicador=2)
# ------------------------------------------------------------------
hours_per_day = 24
num_days_test = len(y_test_orig) // hours_per_day
np.random.seed(42)
y_test_with_anomalies = y_test_orig.copy()
anomaly_labels = np.zeros(len(y_test_orig), dtype=int)

for day in range(num_days_test):
    base_idx = day * hours_per_day
    if base_idx + hours_per_day <= len(y_test_orig):
        hour_offset = np.random.randint(0, hours_per_day)
        anomaly_idx = base_idx + hour_offset
        y_test_with_anomalies[anomaly_idx] *= 2
        anomaly_labels[anomaly_idx] = 1

print(f"\nAnomalías insertadas: {np.sum(anomaly_labels)} (1 por día)")

# Normalizar anomalías
y_test_norm_anom = (y_test_with_anomalies - min_active) / (max_active -␣
 ↪min_active)

# ------------------------------------------------------------------
# 4. Cálculo de scores (ecuaciones 10 y 11 del artículo)
# ------------------------------------------------------------------
abs_errors = np.abs(y_pred_corrected - y_test_norm_anom)
mean_abs_error = np.mean(abs_errors)
scores = abs_errors / mean_abs_error
scores_norm = (scores - np.min(scores)) / (np.max(scores) - np.min(scores))

# ------------------------------------------------------------------
# 5. Análisis de umbrales
# ------------------------------------------------------------------
thresholds = [0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50, 0.55, 0.60]
results = []
for th in thresholds:
    y_pred = (scores_norm > th).astype(int)
    acc = accuracy_score(anomaly_labels, y_pred)
    prec = precision_score(anomaly_labels, y_pred, zero_division=0)
    rec = recall_score(anomaly_labels, y_pred, zero_division=0)
    f1 = f1_score(anomaly_labels, y_pred, zero_division=0)
```

```
    detections = y_pred.sum()
    results.append([th, acc, prec, rec, f1, detections])

df_thresholds = pd.DataFrame(results, columns=['Umbral', 'Accuracy',␣
 ↪'Precision', 'Recall', 'F1', 'Detecciones'])

print("\n" + "="*80)
print("TABLA 1: MÉTRICAS PARA DIFERENTES UMBRALES")
print("="*80)
print(df_thresholds.to_string(index=False))

# Mejores valores
best_precision = df_thresholds.loc[df_thresholds['Precision'].idxmax()]
best_recall = df_thresholds.loc[df_thresholds['Recall'].idxmax()]
best_f1 = df_thresholds.loc[df_thresholds['F1'].idxmax()]

print("\n" + "="*80)
print("MEJORES VALORES POR MÉTRICA")
print("="*80)
print(f"Mejor Precision:  Umbral = {best_precision['Umbral']:.2f}  →  Precision␣
 ↪= {best_precision['Precision']:.4f}")
print(f"Mejor Recall:     Umbral = {best_recall['Umbral']:.2f}  →  Recall    =␣
 ↪{best_recall['Recall']:.4f}")
print(f"Mejor F1-Score:   Umbral = {best_f1['Umbral']:.2f}  →  F1        =␣
 ↪{best_f1['F1']:.4f}")

# ------------------------------------------------------------------
# 6. Umbral del artículo (0.45)
# ------------------------------------------------------------------
threshold_article = 0.45
y_pred_article = (scores_norm > threshold_article).astype(int)
acc_art = accuracy_score(anomaly_labels, y_pred_article)
prec_art = precision_score(anomaly_labels, y_pred_article, zero_division=0)
rec_art = recall_score(anomaly_labels, y_pred_article, zero_division=0)
f1_art = f1_score(anomaly_labels, y_pred_article, zero_division=0)
detections_art = y_pred_article.sum()

print("\n" + "="*80)
print(f"RESULTADOS CON EL UMBRAL DEL ARTÍCULO ({threshold_article})")
print("="*80)
print(f"Accuracy : {acc_art:.4f}")
print(f"Precision: {prec_art:.4f}")
print(f"Recall   : {rec_art:.4f}")
print(f"F1-Score : {f1_art:.4f}")
print(f"RMSE     : {rmse:.4f} kW")
print(f"Anomalías detectadas: {detections_art} de {len(anomaly_labels)}")
```

```python
# Matriz de confusión
tp = np.sum((anomaly_labels == 1) & (y_pred_article == 1))
fp = np.sum((anomaly_labels == 0) & (y_pred_article == 1))
fn = np.sum((anomaly_labels == 1) & (y_pred_article == 0))
tn = np.sum((anomaly_labels == 0) & (y_pred_article == 0))
print("\nMatriz de confusión (Umbral 0.45):")
print(f"  VP: {tp}, FP: {fp}, FN: {fn}, VN: {tn}")
print("="*80)

# ----------------------------------------------------------------------
# 7. Tabla comparativa con el artículo (Tabla 2)
# ----------------------------------------------------------------------
print("\n" + "="*80)
print("TABLA 2: COMPARACIÓN CON LOS VALORES REPORTADOS EN EL ARTÍCULO")
print("="*80)
comparison = pd.DataFrame({
    'Métrica': ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'RMSE (kW)'],
    'Artículo (Tabla 2)': [0.970, 0.800, 0.660, 0.720, 0.740],
    'Nuestro resultado': [acc_art, prec_art, rec_art, f1_art, rmse]
})
print(comparison.to_string(index=False))
print("="*80)

# Guardar resultados
df_thresholds.to_csv(f"{BASE_DIR}/threshold_analysis.csv", index=False)
comparison.to_csv(f"{BASE_DIR}/comparison_article.csv", index=False)
print("\nResultados guardados en 'threshold_analysis.csv' y 'comparison_article.
  ↪csv'.")
```

```
Cargando datos…
Cargando modelo Transformer…
Cargando K-Means (optimización)…
Centroides de K-Means (en espacio de y): [0.36025018 0.66665284 0.54608692
0.18519321 0.952794   0.44980085
 0.78311995 0.04510666 0.2752222  0.49860481]
Modelo LSTM cargado.

Generando predicciones con Transformer + K-Means…
RMSE (Transformer + K-Means) en escala original: 13.1421 kW
RMSE (LSTM) en escala original: 13.8572 kW

Anomalías insertadas: 69 (1 por día)

================================================================================
TABLA 1: MÉTRICAS PARA DIFERENTES UMBRALES
================================================================================
 Umbral  Accuracy  Precision   Recall       F1  Detecciones
```

```
   0.10  0.343470   0.048951 0.811594 0.092333         1144
   0.15  0.490161   0.052392 0.666667 0.097149          878
   0.20  0.645200   0.064570 0.565217 0.115899          604
   0.25  0.806202   0.112121 0.536232 0.185464          330
   0.30  0.912343   0.232877 0.492754 0.316279          146
   0.35  0.952892   0.430556 0.449275 0.439716           72
   0.40  0.965414   0.622222 0.405797 0.491228           45
   0.45  0.974955   1.000000 0.391304 0.562500           27
   0.50  0.972570   1.000000 0.333333 0.500000           23
   0.55  0.968992   1.000000 0.246377 0.395349           17
   0.60  0.966011   1.000000 0.173913 0.296296           12

================================================================================
MEJORES VALORES POR MÉTRICA
================================================================================
Mejor Precision:  Umbral = 0.45  →  Precision = 1.0000
Mejor Recall:     Umbral = 0.10  →  Recall    = 0.8116
Mejor F1-Score:   Umbral = 0.45  →  F1        = 0.5625

================================================================================
RESULTADOS CON EL UMBRAL DEL ARTÍCULO (0.45)
================================================================================
Accuracy : 0.9750
Precision: 1.0000
Recall   : 0.3913
F1-Score : 0.5625
RMSE     : 13.1421 kW
Anomalías detectadas: 27 de 1677

Matriz de confusión (Umbral 0.45):
  VP: 27, FP: 0, FN: 42, VN: 1608
================================================================================

================================================================================
TABLA 2: COMPARACIÓN CON LOS VALORES REPORTADOS EN EL ARTÍCULO
================================================================================
  Métrica  Artículo (Tabla 2)  Nuestro resultado
 Accuracy                0.97           0.974955
Precision                0.80           1.000000
   Recall                0.66           0.391304
 F1-Score                0.72           0.562500
RMSE (kW)                0.74          13.142051
================================================================================

Resultados guardados en 'threshold_analysis.csv' y 'comparison_article.csv'.
```

[23]:
```python
# ================================================================
# GENERACIÓN DE GRÁFICAS (FIGURAS 3, 4 Y 5)
# Adaptado a los archivos de /content
# ================================================================

import os
os.chdir("/content")

import numpy as np
import tensorflow as tf
import joblib
import pickle
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

BASE_DIR = "/content"
tf.keras.config.enable_unsafe_deserialization()

# ----------------------------------------------------------------
# 1. Cargar datos
# ----------------------------------------------------------------
print("Cargando datos...")
data = np.load(f"{BASE_DIR}/ds3_preprocessed_rms.npz", allow_pickle=True)
X_test = data["X_test"]
y_test_norm = data["y_test"].flatten()

scaler_y = joblib.load(f"{BASE_DIR}/scaler_y.pkl")
min_active = scaler_y.data_min_[0]
max_active = scaler_y.data_max_[0]

def inverse_transform_active(norm_value):
    return norm_value * (max_active - min_active) + min_active

# ----------------------------------------------------------------
# 2. Cargar modelos
# ----------------------------------------------------------------
print("Cargando modelo Transformer...")
transformer = tf.keras.models.load_model(
    f"{BASE_DIR}/best_transformer.keras",
    compile=False,
    safe_mode=False
)

print("Cargando K-Means...")
kmeans_opt = joblib.load(f"{BASE_DIR}/kmeans_model.pkl")
centroids = kmeans_opt.cluster_centers_.flatten()
```

```
print("Cargando modelo LSTM...")
lstm_model = tf.keras.models.load_model(
    f"{BASE_DIR}/lstm_model.keras",
    compile=False,
    safe_mode=False
)

# ------------------------------------------------------------------
# 3. Predicciones
# ------------------------------------------------------------------
transformer_preds_norm = transformer.predict(X_test, verbose=0).flatten()
kmeans_corrected_norm = np.array([
    centroids[np.argmin(np.abs(transformer_preds_norm[i] - centroids))]
    for i in range(len(transformer_preds_norm))
])
lstm_preds_norm = lstm_model.predict(X_test, verbose=0).flatten()

y_test_orig = inverse_transform_active(y_test_norm)
our_preds_orig = inverse_transform_active(kmeans_corrected_norm)
lstm_preds_orig = inverse_transform_active(lstm_preds_norm)

# ------------------------------------------------------------------
# 4. Figura 3: Pérdida de entrenamiento
# ------------------------------------------------------------------
try:
    try:
        history = joblib.load(f"{BASE_DIR}/training_history.pkl")
    except Exception:
        with open(f"{BASE_DIR}/training_history.pkl", "rb") as f:
            history = pickle.load(f)

    plt.figure(figsize=(10,5))
    plt.plot(history['loss'], label='Train loss', linewidth=1.5)
    plt.plot(history['val_loss'], label='Validation loss', linewidth=1.5)
    plt.xlabel('Epoch', fontsize=12)
    plt.ylabel('Loss (MSE)', fontsize=12)
    plt.title('Figure 3: Train and test loss over the 300 epochs', fontsize=14)
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig(f"{BASE_DIR}/figure3_loss.png", dpi=150)
    plt.show()
    print("Figura 3 guardada como 'figure3_loss.png'")
except Exception as e:
    print(f"No se pudo generar Figura 3: {e}")
```

```
# ------------------------------------------------------------------
# 5. Figura 4: Comparación de predicciones (3 días)
# ------------------------------------------------------------------
start_idx = 0
end_idx = start_idx + 72
hours = np.arange(start_idx, end_idx)

plt.figure(figsize=(12,5))
plt.plot(hours, y_test_orig[start_idx:end_idx], 'b-', label='Real data',␣
 ↪linewidth=1.5)
plt.plot(hours, lstm_preds_orig[start_idx:end_idx], 'g-', label='LSTM',␣
 ↪linewidth=1.5)
plt.plot(hours, our_preds_orig[start_idx:end_idx], 'r-', label='Our model␣
 ↪(Transformer + K-Means)', linewidth=1.5)
plt.xlabel('Hour (test set)', fontsize=12)
plt.ylabel('Global active power (kW)', fontsize=12)
plt.title('Figure 4: Comparison of predicted values with real data (3 days)',␣
 ↪fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(f"{BASE_DIR}/figure4_comparison.png", dpi=150)
plt.show()
print("Figura 4 guardada como 'figure4_comparison.png'")

# ------------------------------------------------------------------
# 6. Inyección de anomalías y scores
# ------------------------------------------------------------------
hours_per_day = 24
num_days_test = len(y_test_orig) // hours_per_day
np.random.seed(42)
y_test_orig_with_anomalies = y_test_orig.copy()
anomaly_labels = np.zeros(len(y_test_orig), dtype=int)

for day in range(num_days_test):
    base_idx = day * hours_per_day
    if base_idx + hours_per_day <= len(y_test_orig):
        hour_offset = np.random.randint(0, hours_per_day)
        anomaly_idx = base_idx + hour_offset
        y_test_orig_with_anomalies[anomaly_idx] *= 2
        anomaly_labels[anomaly_idx] = 1

y_test_norm_with_anomalies = (y_test_orig_with_anomalies - min_active) /␣
 ↪(max_active - min_active)

abs_errors = np.abs(kmeans_corrected_norm - y_test_norm_with_anomalies)
mean_abs_error = np.mean(abs_errors)
```

```
scores = abs_errors / mean_abs_error
scores_norm = (scores - np.min(scores)) / (np.max(scores) - np.min(scores))

threshold = 0.45

# ----------------------------------------------------------------
# 7. Figura 5: Scores y anomalías (2500 horas)
# ----------------------------------------------------------------
max_hours = min(2500, len(scores_norm))
time_axis = np.arange(max_hours)
anom_in_range = np.where((scores_norm > threshold) & (np.
 ↪arange(len(scores_norm)) < max_hours))[0]

plt.figure(figsize=(14,5))
plt.plot(time_axis, scores_norm[:max_hours], 'k-', linewidth=0.8,␣
 ↪label='Normalized score')
plt.axhline(y=threshold, color='r', linestyle='--', linewidth=1.5,␣
 ↪label=f'Threshold = {threshold}')
plt.scatter(anom_in_range, scores_norm[anom_in_range], color='magenta', s=30,␣
 ↪label='Anomalies detected')
plt.xlabel('Hour (test set)', fontsize=12)
plt.ylabel('Normalized score', fontsize=12)
plt.title('Figure 5: Scores and anomalies exceeding the threshold for 2500␣
 ↪hours', fontsize=14)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(f"{BASE_DIR}/figure5_scores_anomalies.png", dpi=150)
plt.show()
print("Figura 5 guardada como 'figure5_scores_anomalies.png'")

print("\nTodas las figuras se han generado correctamente.")
```

```
Cargando datos…
Cargando modelo Transformer…
Cargando K-Means…
Cargando modelo LSTM…
```

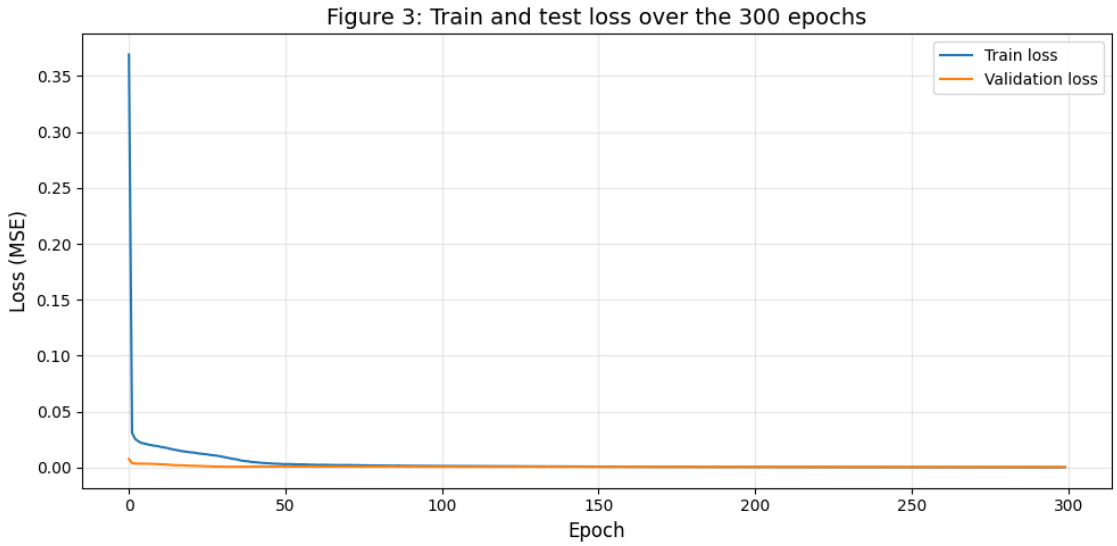


Figura 3 guardada como 'figure3_loss.png'

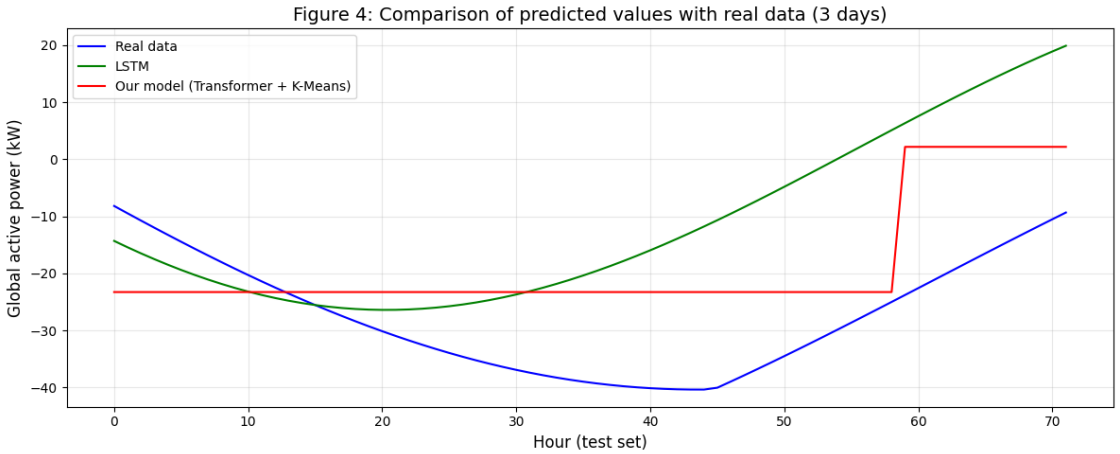
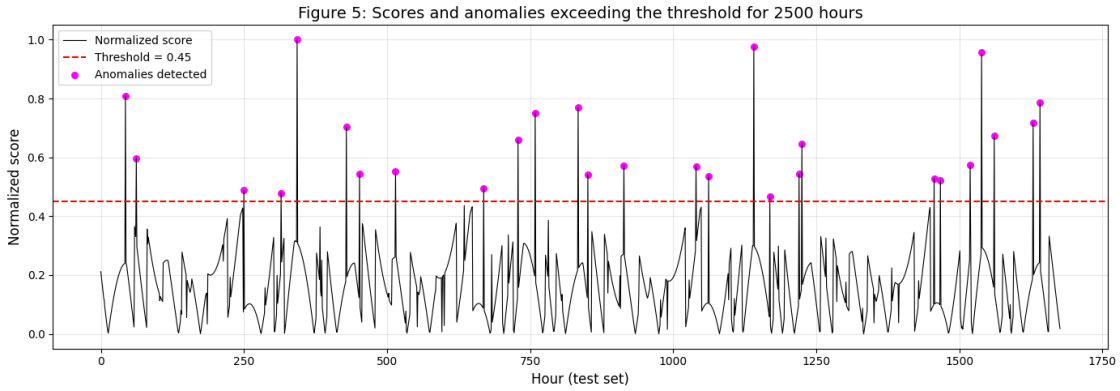


Figura 4 guardada como 'figure4_comparison.png'




```
Figura 5 guardada como 'figure5_scores_anomalies.png'

Todas las figuras se han generado correctamente.
```